ChainForge API Reference Guide

API version v1 · October 2026 · Companion to the ChainForge Developer Guide

> **Note:** ChainForge is a fictional blockchain platform created for documentation practice. All endpoints, domains, keys, addresses and identifiers in this guide are invented and are not real services.

Contents at a Glance

1. Introduction
2. Authentication
3. Common API Behavior
4. Status and Network
5. Accounts
6. Transactions
7. Gas
8. Smart Contracts
9. Tokens and NFTs
10. Webhooks
11. Usage
12. Real-Time Subscriptions (WebSocket)
13. Models (Schemas)
14. Error Catalog
15. Reference Tables
16. Changelog and Deprecations

Appendices: A. Code samples · B. Postman collection and OpenAPI download · C. Glossary

---

1. Introduction

1.1 About This Reference

This guide is the exhaustive, lookup-style description of the ChainForge REST API and its real-time interfaces. It lists every endpoint, every parameter, every field of every request and response, and every error you can receive. It is designed to be searched and scanned rather than read from beginning to end.

If you are new to ChainForge, or you want to learn how to accomplish a task such as taking a payment or minting a token, start with the **ChainForge Developer Guide**. That guide explains *why* and *how*; this one defines *exactly what*. The two are cross-referenced: each task in the Developer Guide's Appendix D points to the endpoints documented here.

What this reference covers

Area	Chapters
Getting authorized and understanding shared behavior	2 and 3
Endpoint reference, grouped by resource	4 to 11
WebSocket subscriptions	12
Object definitions, errors and enumerations	13 to 15
Change history	16

What this reference does not cover

- Concepts such as gas, finality or smart contract design. See Developer Guide chapters 3 and 6.
- Step-by-step tutorials. See Developer Guide chapter 10.
- The JSON-RPC endpoint. ChainForge nodes expose a standard Ethereum-compatible JSON-RPC interface, documented by the Ethereum JSON-RPC specification. Only the platform REST API is documented here.

The machine-readable source

This guide is generated from and stays consistent with the ChainForge OpenAPI 3.0 specification (see Appendix B). If this guide and the specification ever disagree, the specification is authoritative and you should report the discrepancy.

1.2 How to Read an Endpoint Entry

Every endpoint in chapters 4 to 11 follows the same layout, so you can find what you need quickly.

Part of the entry	What it tells you
Heading	The HTTP method and path, for example `GET /accounts/{address}/balance`
Summary	One sentence describing what the endpoint does
Required scope	The OAuth scope or API key permission needed to call it (see section 2.3)
Description	Behavior, side effects, idempotency, finality and anything surprising
Path parameters	Values that appear inside the URL, always required
Query parameters	Optional or required values after the `?`
Headers	Request headers beyond standard authentication
Request body	The JSON payload, with a field table and an example
Responses	Each status code, its meaning, a schema reference and an example
Errors	Error codes specific to this endpoint (the full catalog is in chapter 14)
Code samples	`curl`, JavaScript and Python

Notation used in tables

Symbol or term	Meaning
Required	The request is rejected with `400` or `422` if the field is missing
Optional	May be omitted; a default applies where stated
`string<address>`	A 42-character hexadecimal address starting with `0x`
`string<hash>`	A 66-character hexadecimal hash starting with `0x`
`string<uint256>`	A non-negative integer sent as a decimal **string** (to avoid numeric precision loss)
`string<datetime>`	An ISO 8601 timestamp in UTC, such as `2026-10-18T09:30:12Z`
`string<hex>`	Hexadecimal bytes starting with `0x`
`{id}`	A placeholder in a path that you replace with a real value

1.3 Base URLs and Environments

The REST API has a single host. **Your API key's prefix decides which network a request reaches.**

Item	Value
REST base URL	`https://api.chainforge.example/v1`
OAuth token server	`https://auth.chainforge.example`
WebSocket URL (mainnet)	`wss://ws.chainforge.example/v1`
WebSocket URL (Ember testnet)	`wss://ws.ember.chainforge.example/v1`
Local devnet (REST and RPC)	`http://127.0.0.1:8545`

Key prefix	Network	Chain ID	Native token
`cfk_live_`	Forge Mainnet	7700	FORGE

Key prefix	Network	Chain ID	Native token
`cfk_test_`	Ember Testnet	7701	tFORGE
(devnet; no key needed)	Local Devnet	7702	dFORGE

Because the host is shared, a `cfk_test_` key can never reach mainnet and a `cfk_live_` key can never reach Ember. Every response that depends on network state includes a `network` field so your logs are unambiguous.

Example: the same path, two networks

```
# Ember (testnet) - key starts with cfk_test_
curl https://api.chainforge.example/v1/status -H "Authorization: Bearer
cfk_test_ABC123"

# Mainnet - key starts with cfk_live_
curl https://api.chainforge.example/v1/status -H "Authorization: Bearer
cfk_live_XYZ789"
```

The first returns `"network": "ember"` and `"chainId": 7701`. The second returns `"network": "mainnet"` and `"chainId": 7700`.

1.4 Conventions

Data formats

Topic	Convention
Transport	HTTPS only (TLS 1.2 or later). HTTP requests are refused.
Encoding	UTF-8 JSON for requests and responses. Send `Content-Type: application/json` on requests with a body.
Field names	`camelCase` in JSON.
Timestamps	ISO 8601 in UTC with a `Z` suffix, to the second: `2026-10-18T09:30:12Z`.
Booleans	JSON `true` and `false`.

Topic	Convention
Null and absent fields	A field that has no value is either `null` or omitted; the documentation states which. Clients must tolerate both.
Unknown fields	Clients must **ignore** response fields they do not recognize. New fields may appear within a version.

Amounts and numbers

All amounts of native tokens and token units are **integers in the smallest unit**, sent and returned as **decimal strings**.

Unit	Relationship
1 FORGE	1,000,000,000,000,000,000 sparks (10^18)
1 gwei-equivalent	1,000,000,000 sparks (10^9)
Token units	Defined by the token's `decimals` (for example, 6 decimals: 1.00 token = `"1000000"`)

```
{ "value": "5000000000000000000" }
```

That is 5 FORGE. Never send amounts as JSON numbers or as decimals such as `"5.0"`. Doing so returns `400 invalid_amount`.

Small integers that cannot exceed the safe numeric range (such as `blockNumber`, `nonce`, `limit` and `chainId`) are returned as ordinary JSON numbers.

Identifiers

Kind	Format	Example
Address	`0x` + 40 hex characters. Accepted in any case; mixed case is checksum-validated. Responses return checksummed addresses.	`0x71C7656EC7ab88b098defB751B7401B5f6d8976F`
Transaction hash	`0x` + 64 hex characters	`0x9f3c5d8a1b7e44c2a09d3f6b8e1c2d4a5b6c7d8e9f001122334455667788 99aa`
Block hash	`0x` + 64 hex characters	`0xa1b2c3d4...`
Platform resource IDs	Short prefix + underscore + sortable unique string	`txn_01JC7X3M9Q`, `wh_01JC9K4TQZ`, `evt_01JC9M7N2X`, `req_7H2K9PQ`

Platform IDs are opaque. Treat them as strings of up to 40 characters, never parse them, and never assume a particular length.

Request IDs

Every response carries an `X-Request-Id` header (for example `req_7H2K9PQ`). The same value appears as `requestId` inside any error body. Log it and include it when contacting support.

---

2. Authentication

All endpoints except the OAuth token endpoint require authentication. There are two methods, and both put a **bearer token** in the `Authorization` header.

```
Authorization: Bearer <token>
```

2.1 API Keys

An **API key** is a long-lived secret created in the Developer Console (Project → API Keys → Create key).

Property	Detail
Format	`cfk_test_` or `cfk_live_` followed by 32 alphanumeric characters
Shown	Once, at creation. It cannot be retrieved later.
Permissions	`read`, `write`, `admin`, chosen when the key is created
Optional restrictions	Allowed IP addresses (CIDR ranges)
Lifetime	Until revoked or rotated

Permission levels

Permission	Grants these scopes (see 2.3)
`read`	`accounts:read`, `transactions:read`, `contracts:read`
`write`	Everything in `read`, plus `transactions:write`, `contracts:write`
`admin`	Everything in `write`, plus `webhooks:manage` and `admin`

Example request

```
curl
https://api.chainforge.example/v1/accounts/0x71C7656EC7ab88b098defB751B7401B5
f6d8976F \
  -H "Authorization: Bearer cfk_test_8hG2kLm9PqRsT4vWxYz1AbCdEfGhJkLm"
```

Failure responses

Situation	Status	Error code
Header missing or malformed	`401`	`missing_credentials`
Key is wrong, revoked or from another project	`401`	`invalid_api_key`
Key's permission is too low for the endpoint	`403`	`insufficient_scope`
Request came from an IP not on the key's allow-list	`403`	`ip_not_allowed`

> **Warning:** API keys must only be used from servers you control. Never embed one in browser or mobile code. See Developer Guide section 4.2.

2.2 OAuth 2.0 (Client Credentials)

OAuth issues short-lived access tokens with fine-grained scopes. Use it when your security policy prohibits long-lived secrets or when you need to restrict a service to specific operations.

How it works

1. In the Console, create an **OAuth client** under Project → OAuth Clients. You receive a `client_id` and a `client_secret` (shown once).
2. Your service calls `POST /oauth/token` on the auth server (section 2.4), asking for the scopes it needs.
3. The server returns an access token valid for **3,600 seconds**.
4. Send that token as a bearer token on API requests, exactly like an API key.
5. Request a new token shortly before the old one expires. Cache the token in memory and reuse it.

```
Your service --(client_id + client_secret)-->
auth.chainforge.example/oauth/token
             <--(access_token, expires_in=3600)--
Your service --(Authorization: Bearer access_token)-->
api.chainforge.example/v1/...
```

Property	Detail
Supported grant	`client_credentials`

Property	Detail
Token format	Signed JWT (treat as opaque; do not parse it)
Token lifetime	3,600 seconds
Refresh tokens	Not issued for client credentials. Request a new access token.
Environment	An OAuth client belongs to either the test or the live environment

> **Note:** The authorization code flow with PKCE is available for applications that act on behalf of end users, and is described in the Console documentation. This reference covers machine-to-machine access.

2.3 Scopes by Endpoint

A request succeeds only if the credential holds the required scope. API keys receive scopes according to their permission level (section 2.1). OAuth tokens receive exactly the scopes requested at token time (and allowed for the client).

Scope	Allows
`accounts:read`	Read accounts, balances, nonces, account transaction lists and NFTs
`transactions:read`	Read transactions and gas estimates
`transactions:write`	Prepare and submit transactions, speed up and cancel
`contracts:read`	Read contract metadata, make read-only calls, simulate, query events, read tokens
`contracts:write`	Deploy and register contracts, create tokens, register accounts
`webhooks:manage`	Create, read, update, delete and test webhooks; redeliver events
`admin`	Read usage; manage project-level settings

Endpoint-to-scope summary

Endpoint	Scope
`GET /status`, `GET /blocks/*`	Any valid credential
`POST /accounts`	`contracts:write` is **not** required; requires `accounts:read` and the `write` permission; OAuth: `accounts:read` plus `transactions:write`
`GET /accounts/**`	`accounts:read`
`POST /transactions/prepare`, `POST /transactions`, speed-up, cancel	`transactions:write`
`GET /transactions/{id}`	`transactions:read`
`POST /gas/estimate`	`transactions:read`
`POST /contracts`, `POST /contracts/{address}/register`, `POST /tokens`	`contracts:write`

Endpoint	Scope
`GET /contracts/**`, `POST /contracts/{address}/call`, `POST /contracts/{address}/simulate`, `GET /tokens/**`	`contracts:read`
`/webhooks/**`	`webhooks:manage`
`GET /usage`	`admin`

2.4 `POST /oauth/token`

Summary: Exchange OAuth client credentials for a short-lived access token.

Server: `https://auth.chainforge.example` (not the REST host)

Authentication: none (the credentials are in the request body)

Content type: `application/x-www-form-urlencoded`

Request body

Field	Type	Required	Description
`grant_type`	string	**Required**	Must be `client_credentials`
`client_id`	string	**Required**	Your OAuth client ID
`client_secret`	string	**Required**	Your OAuth client secret
`scope`	string	*Optional*	Space-separated scopes. If omitted, all scopes allowed for the client are granted.

Example request

```
curl -X POST https://auth.chainforge.example/oauth/token \
  -d grant_type=client_credentials \
  -d client_id=cfc_01JC8A5W2D \
  -d client_secret=YOUR_CLIENT_SECRET \
  -d scope="transactions:read transactions:write"
```

Responses

`200 OK` : token issued

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "scope": "transactions:read transactions:write"
}
```

Field	Type	Description
`access_token`	string	The bearer token
`token_type`	string	Always `Bearer`
`expires_in`	integer	Lifetime in seconds
`scope`	string	The scopes actually granted, which may be fewer than requested

`400 Bad Request` : invalid request

```
{ "error": "invalid_request", "error_description": "grant_type is required."
}
```

`401 Unauthorized` : bad client credentials

```
{ "error": "invalid_client", "error_description": "Client authentication
failed." }
```

`400 Bad Request` : scope not allowed

```
{ "error": "invalid_scope", "error_description": "Scope 'admin' is not
allowed for this client." }
```

> **Note:** The token endpoint uses the standard OAuth 2.0 error format (`error` and `error_description`) rather than the ChainForge error object described in section 3.2.

Code samples

```
const res = await fetch("https://auth.chainforge.example/oauth/token", {
  method: "POST",
  headers: { "Content-Type": "application/x-www-form-urlencoded" },
  body: new URLSearchParams({
    grant_type: "client_credentials",
    client_id: process.env.CF_CLIENT_ID,
    client_secret: process.env.CF_CLIENT_SECRET,
    scope: "transactions:read transactions:write",
  }),
});
const { access_token, expires_in } = await res.json();
```

```
import os, requests

r = requests.post("https://auth.chainforge.example/oauth/token", data={
    "grant_type": "client_credentials",
    "client_id": os.environ["CF_CLIENT_ID"],
    "client_secret": os.environ["CF_CLIENT_SECRET"],
    "scope": "transactions:read transactions:write",
})
r.raise_for_status()
token = r.json()["access_token"]
```

The official SDKs perform this exchange and token caching automatically when you pass `clientId` and `clientSecret` instead of `apiKey`.

---

3. Common API Behavior

The rules in this chapter apply to every endpoint, so the endpoint entries do not repeat them.

3.1 Request and Response Format

Requests

Rule	Detail
Methods	`GET` reads; `POST` creates or performs an action; `PATCH` partially updates; `DELETE` removes. `PUT` is not used.
Body	JSON object. Send `Content-Type: application/json`. A `GET` or `DELETE` request must not have a body.
Maximum body size	1 MB (contract deployment bytecode may be up to 256 KB within that limit)
Query strings	URL-encode values. Arrays use comma-separated values: `?addresses=0xA...,0xB...`
Compression	`Accept-Encoding: gzip` is supported and recommended for large responses

Successful responses

Status	Used when
`200 OK`	A read or an action completed and returned data
`201 Created`	A resource was created. A `Location` header holds its URL.
`202 Accepted`	The work was accepted but completes asynchronously (for example, a submitted transaction)
`204 No Content`	Success with no body (for example, `DELETE`)

Response envelope

Single resources are returned as the object itself, with no wrapper. Lists use a consistent envelope:

```
{
  "data": [ { "...": "..." } ],
  "nextCursor": "eyJvIjoyNX0"
}
```

Standard response headers

Header	Meaning
`X-Request-Id`	Unique ID of this request
`X-ChainForge-Network`	`mainnet`, `ember` or `devnet`
`X-RateLimit-Limit`	Requests per second allowed on your plan
`X-RateLimit-Remaining`	Requests remaining in the current window
`X-RateLimit-Reset`	Seconds until the window resets
`Retry-After`	On `429` and `503`: seconds to wait before retrying
`Deprecation`, `Sunset`	Present on deprecated endpoints (section 3.7)

3.2 Errors and the Error Object

Any non-`2xx` response from the REST API (except the OAuth token endpoint) returns the same JSON structure:

```
{
  "error": {
    "code": "insufficient_funds",
    "message": "The sender balance is too low to cover value plus maximum
fee.",
    "details": {
      "required": "5063000000000000000",
      "available": "4200000000000000000"
    },
    "requestId": "req_5GT7M1K",
    "docUrl": "https://docs.chainforge.example/errors#insufficient_funds"
  }
}
```

Field	Type	Description
`code`	string	Stable machine-readable identifier. **Write code that branches on this field.**
`message`	string	Human-readable explanation. Wording may change; do not match on it.
`details`	object	*Optional.* Structured information. Its keys depend on `code`.
`requestId`	string	The request's unique ID
`docUrl`	string	Link to the catalog entry in chapter 14

Validation errors

When several input fields are invalid, `details.fields` lists each problem:

```
{
  "error": {
    "code": "validation_failed",
    "message": "2 fields failed validation.",
    "details": {
      "fields": [
        { "field": "to", "issue": "Not a valid address." },
        { "field": "value", "issue": "Must be a non-negative integer string."
}
      ]
    },
    "requestId": "req_9ZQ2X4D"
  }
}
```

Which status for which problem

Status	Meaning	Retry the same request?
`400`	The request is malformed	No
`401`	Missing or invalid credentials	No

Status	Meaning	Retry the same request?
`403`	Authenticated but not permitted	No
`404`	Resource not found	Only if it was created moments ago (section 14.1)
`409`	Conflicts with current state	Depends on `code`
`422`	Well-formed but semantically invalid, or the call would revert	No
`429`	Rate limit exceeded	Yes, after `Retry-After`
`5xx`	Platform error	Yes, with exponential backoff

The complete list of codes is in chapter 14.

3.3 Pagination (Cursor-Based)

Endpoints that return collections use **cursor pagination**, which stays correct even when new items arrive while you are paging.

Query parameters

Parameter	Type	Default	Description
`limit`	integer	25	Number of items per page, from 1 to 100
`cursor`	string	none	Opaque cursor from the previous response's `nextCursor`. Omit for the first page.

Response fields

Field	Description
`data`	The array of items for this page
`nextCursor`	Pass this as `cursor` to get the next page. `null` means there are no more pages.

Walk-through

Request 1

```
curl
"https://api.chainforge.example/v1/accounts/0x71C7656EC7ab88b098defB751B7401B
5f6d8976F/transactions?limit=2" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
{
  "data": [ { "id": "txn_01JC9M6A1B" }, { "id": "txn_01JC7X3M9Q" } ],
  "nextCursor": "eyJvIjoyfQ"
}
```

Request 2 (use the cursor, keep the same filters and `limit`)

```
curl "https://api.chainforge.example/v1/accounts/0x71C7.../transactions?
limit=2&cursor=eyJvIjoyfQ" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
{
  "data": [ { "id": "txn_01JC7W2K8N" } ],
  "nextCursor": null
}
```

Rules

1. Treat the cursor as an opaque string. Its format can change.
2. A cursor is valid for 24 hours and only with the same filters that produced it. Changing filters with an old cursor returns `400 invalid_cursor`.
3. The default sort order is **newest first** unless an endpoint states otherwise.
4. Do not assume a full page means more data. Check `nextCursor`.
5. The SDKs' async iterators follow cursors for you.

3.4 Idempotency

Network failures create uncertainty: if a request times out, you cannot tell whether it was processed. **Idempotency keys** make it safe to retry operations that change state.

Aspect	Detail
Header	`Idempotency-Key: <string>`
Format	8 to 64 characters (a UUID is recommended)
Applies to	Every `POST` endpoint that creates or submits something (transactions, contracts, tokens, webhooks, accounts)
Required on	`POST /transactions` and `POST /tokens`; recommended elsewhere
Retention	24 hours from the first request

Behavior

Situation	Result
First use of a key	Processed normally. The response is stored.
Retry with the **same key and same body**	The original response is replayed with the header `Idempotent-Replayed: true`. Nothing is processed again.
Same key, **different body**	`409 idempotency_conflict`
Same key while the first request is still in progress	`409 idempotency_in_progress`. Retry after a short delay.
Key expired (older than 24 hours)	Treated as a new request

Example

```
curl -X POST https://api.chainforge.example/v1/transactions \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: 6f1d2c9a-3b7e-4d55-9a21-0e8c4b7a1f90" \
  -d '{ "signedTransaction": "0xf86c0a85..." }'
```

If this call times out, send the identical request again. You will get the same transaction back rather than a duplicate.

> **Note:** Only responses with status `2xx` and `4xx` that reflect a final decision are stored. A `5xx` response is not stored, so retrying with the same key is always safe.

3.5 Rate Limits and Headers

Limits are applied **per project and per second**, with a daily quota in addition.

Plan	Requests per second	Requests per day	Concurrent WebSocket subscriptions
Free	10	100,000	5
Growth	50	5,000,000	50
Enterprise	Custom	Custom	Custom

All responses include `X-RateLimit-Limit`, `X-RateLimit-Remaining` and `X-RateLimit-Reset`.

When a limit is exceeded the API returns `429 Too Many Requests` with `Retry-After`:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 2
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1
```

```
{
  "error": {
    "code": "rate_limited",
    "message": "Rate limit of 10 requests per second exceeded.",
    "requestId": "req_7H2K9PQ"
  }
}
```

Endpoint cost. Most endpoints count as one request. A few cost more:

Endpoint	Cost
`GET /accounts/balances`	1 request per 10 addresses, rounded up
`GET /contracts/{address}/events` with `limit` above 50	2 requests
`POST /contracts/{address}/simulate`	2 requests

Recommended client behavior: wait `Retry-After` seconds, double the wait on repeated failures (1, 2, 4, 8 seconds), add random jitter and cap retries (five attempts is typical). See Developer Guide section 4.4 for a code sample.

3.6 Finality and Block Selectors

Many read endpoints let you choose **which state of the chain** to read, using the `block` query parameter.

Value	Meaning	When to use
`latest` *(default)*	The most recent included block. Fast, but might be superseded in rare cases before it is final.	Interfaces that tolerate small corrections
`finalized`	The most recent finalized block (about 4 seconds behind `latest`). Never reversed.	Accounting, order fulfilment, reconciliation
`pending`	Latest state plus transactions currently in the mempool. Supported only where stated (for example, nonces).	Computing the next nonce
`<number>`	A specific historical block, such as `4820141`	Audits, debugging, reproducing past results

Every response that reads chain state includes the `blockNumber` it was read at, so you always know what you saw.

```
curl "https://api.chainforge.example/v1/accounts/0x71C7.../balance?
block=finalized" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

A historical block older than 128 blocks may require an archive-enabled plan; otherwise the API returns `422 state_unavailable`.

3.7 Versioning and Deprecation

Policy	Detail
Version in the URL	`https://api.chainforge.example/v1/...`
Compatible changes within a version	New endpoints, new optional request fields, new response fields, new enum values, new error codes. Clients must tolerate these.
Breaking changes	Removing or renaming fields or endpoints, changing types or meanings, making an optional field required. These ship **only** in a new major version (`/v2`).
Support window	A previous major version is supported for at least **12 months** after its successor is generally available
Deprecation notice	Announced in the changelog (chapter 16), emailed to project administrators and signaled on responses

Deprecated endpoints return two headers:

```
Deprecation: true
Sunset: Wed, 31 Mar 2027 00:00:00 GMT
Link: <https://docs.chainforge.example/changelog#legacy-search>;
rel="deprecation"
```

Writing a forward-compatible client

1. Ignore unknown response fields.
2. Do not depend on the order of fields or array elements unless documented.
3. Handle unknown enum values gracefully (log and continue).

4. Branch on `error.code`, not on `error.message`.
5. Pin your SDK to a minor version and test upgrades on Ember.

---

4. Status and Network

These endpoints report the health and position of the network your credential is connected to. They need no particular scope, only a valid credential, which makes them ideal for connectivity checks, startup validation and dashboards.

4.1 `GET /status`

Summary: Return the environment, network, latest block and API health for the credential's network.

Required scope: any valid credential

Description

Use this endpoint to confirm that your key works and that it is connected to the network you intend. Compare `chainId` with the value you expect **at application startup** and refuse to run if they differ. This single check prevents most test-versus-production mistakes.

Parameters

None.

Responses

`200 OK`

```
{
  "environment": "test",
  "network": "ember",
  "chainId": 7701,
  "latestBlock": 4820113,
  "finalizedBlock": 4820111,
  "apiVersion": "v1",
  "status": "operational",
  "time": "2026-10-18T09:30:12Z"
}
```

Field	Type	Description
`environment`	string	`test` or `live`
`network`	string	`ember`, `mainnet` or `devnet`
`chainId`	integer	Chain ID of the network (7700, 7701 or 7702)
`latestBlock`	integer	Number of the most recent included block
`finalizedBlock`	integer	Number of the most recent finalized block
`apiVersion`	string	API version serving the request
`status`	string	`operational`, `degraded` or `maintenance`
`time`	string<datetime>	Server time in UTC. Compare it with your own clock to detect skew (important for webhook signatures).

Status	Code	Meaning
`401`	`invalid_api_key`	Credential rejected
`503`	`service_unavailable`	The platform is temporarily unavailable

Code samples

```
curl https://api.chainforge.example/v1/status \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
const status = await client.status.get();
if (status.chainId !== 7700) throw new Error(`Unexpected chain
${status.chainId}`);
```

```
status = client.status.get()
assert status.chain_id == 7700, f"Unexpected chain {status.chain_id}"
```

4.2 `GET /blocks/{blockId}`

Summary: Retrieve a block by number, by hash or by the keywords `latest` and `finalized`.

Required scope: any valid credential

Path parameters

Name	Type	Description
`blockId`	string	**Required.** A block number (`4820141`), a block hash (`0xa1b2...`), `latest` or `finalized`

Query parameters

Name	Type	Default	Description
`includeTransactions`	boolean	`false`	When `true` , `transactions` contains full transaction objects instead of hashes

Responses

`200 OK`

```
{
  "number": 4820141,
  "hash":
"0xa1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90",
  "parentHash":
"0x77aa5f01c2d3e4b5a69788796a5b4c3d2e1f00112233445566778899aabbccdd",
  "timestamp": "2026-10-18T09:30:14Z",
  "proposer": "0x5B38Da6a701c568545dCfcB03FcB875f56beddC4",
  "transactionCount": 12,
  "transactions": ["0x9f3c5d8a...", "0x4e1d0c7b..."],
  "gasUsed": "412350",
  "gasLimit": "30000000",
  "baseFeePerGas": "2000000000",
  "finalized": true
}
```

Field	Type	Description
`number`	integer	Block height
`hash`, `parentHash`	string<hash>	This block's hash and its predecessor's
`timestamp`	string<datetime>	When the block was produced
`proposer`	string<address>	The validator that proposed the block
`transactionCount`	integer	Number of transactions in the block
`transactions`	array	Transaction hashes, or full objects when `includeTransactions=true`
`gasUsed`, `gasLimit`	string<uint256>	Gas consumed and the block's capacity
`baseFeePerGas`	string<uint256>	Base fee in sparks per gas for this block
`finalized`	boolean	Whether the block is final

Status	Code	Meaning
`400`	`invalid_block_id`	Not a number, hash, `latest` or `finalized`
`404`	`block_not_found`	No such block (for example, a number above the latest)

```
curl "https://api.chainforge.example/v1/blocks/4820141?
includeTransactions=false" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
const block = await client.blocks.get(4820141);
```

4.3 `GET /blocks/finalized`

Summary: Return the most recent finalized block. A convenience alias for `GET /blocks/finalized` as `blockId`.

Required scope: any valid credential

Description

Useful for the *last processed block* bookkeeping described in Developer Guide section 10.3. Everything at or below this block number is permanent.

Responses

`200 OK`: the same block object as section 4.2, with `finalized: true`.

```
curl https://api.chainforge.example/v1/blocks/finalized -H "Authorization:
Bearer $CHAINFORGE_API_KEY"
```

```
const head = await client.blocks.getFinalized();
console.log(head.number);
```

---

5. Accounts

An **account** is an address on the chain. Account endpoints let you register addresses with your project (so the platform indexes their activity and can notify you), read balances and nonces, and list their history.

All chain-state reads on accounts accept the `block` query parameter described in section 3.6.

5.1 `POST /accounts`

Summary: Register an address with your project so the platform tracks its activity.

Required scope: `accounts:read` with the `write` permission (API key) or the `transactions:write` scope (OAuth)

Description

Registering an address is **optional** for reading balances, but required for the platform to index the address's history, deliver `account.received` webhooks and attach your own `metadata`. Registration is idempotent: registering an address that already exists updates its `label` and `metadata` and returns `200` instead of `201`.

Registering does not create a wallet or generate keys. ChainForge never creates or stores private keys.

Headers

Name	Required	Description
`Idempotency-Key`	*Recommended*	See section 3.4

Request body

Field	Type	Required	Description
`address`	string<address>	**Required**	The address to register
`label`	string	*Optional*	Human-readable name, up to 64 characters

Field	Type	Required	Description
`metadata`	object	*Optional*	Up to 16 string keys (each key up to 40 characters, each value up to 256 characters) linking the address to your records. Do not include secrets or personal data.

```
{
  "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "label": "alice-test",
  "metadata": { "customerId": "cust_1042" }
}
```

Responses

`201 Created` (new) or **`200 OK`** (updated)

```
{
  "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "label": "alice-test",
  "type": "eoa",
  "metadata": { "customerId": "cust_1042" },
  "createdAt": "2026-10-18T09:12:44Z",
  "updatedAt": "2026-10-18T09:12:44Z"
}
```

Status	Code	Meaning
`400`	`invalid_address`	Not a valid address or failed checksum
`422`	`metadata_too_large`	More than 16 keys or an oversize key or value
`422`	`account_limit_reached`	Your plan's registered-address limit is reached

```
curl -X POST https://api.chainforge.example/v1/accounts \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -d '{"address":"0x71C7656EC7ab88b098defB751B7401B5f6d8976F","label":"alice-
test","metadata":{"customerId":"cust_1042"}}'
```

```
await client.accounts.register({ address, label: "alice-test", metadata: {
customerId: "cust_1042" } });
```

```
client.accounts.register(address=address, label="alice-test", metadata=
{"customerId": "cust_1042"})
```

5.2 `GET /accounts/{address}`

Summary: Retrieve an account's type, balance, nonce and (for contracts) code information.

Required scope: `accounts:read`

Path parameters

Name	Type	Description
`address`	string<address>	**Required.** The account address

Query parameters

Name	Type	Default	Description
`block`	string	`latest`	Block selector (section 3.6)

Responses

`200 OK` : an externally owned account

```
{
  "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "type": "eoa",
  "balance": "94999937000000000000",
  "nonce": 1,
  "registered": true,
  "label": "alice-test",
  "metadata": { "customerId": "cust_1042" },
  "blockNumber": 4820141
}
```

`200 OK` : a contract account

```
{
  "address": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "type": "contract",
  "balance": "0",
  "nonce": 1,
  "registered": true,
  "codeHash":
"0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470",
  "codeSize": 2418,
  "blockNumber": 4821390
}
```

Field	Type	Description
`address`	string<address>	Checksummed address
`type`	string	`eoa` or `contract`
`balance`	string<uint256>	Native balance in sparks
`nonce`	integer	Transactions sent (EOA) or contracts created (contract)
`registered`	boolean	Whether the address is registered with your project
`label` , `metadata`	string, object	Present only if registered
`codeHash` , `codeSize`	string, integer	Present only for contracts

Field	Type	Description
`blockNumber`	integer	Block the state was read at

An address with no history is **not** an error: the API returns `type: "eoa"`, `balance: "0"` and `nonce: 0`.

```
curl
"https://api.chainforge.example/v1/accounts/0x71C7656EC7ab88b098defB751B7401B
5f6d8976F?block=finalized" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

5.3 `GET /accounts/{address}/balance`

Summary: Return an account's native balance and nonce.

Required scope: `accounts:read`

Path and query parameters

Name	In	Type	Required	Description
`address`	path	string<address>	**Required**	Account address
`block`	query	string	*Optional* (default `latest`)	Block selector

Responses

`200 OK`

```
{
  "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "balance": "94999937000000000000",
  "balanceFormatted": "94.999937",
  "symbol": "tFORGE",
  "blockNumber": 4820141,
  "nonce": 1
}
```

Field	Description
`balance`	Integer sparks as a string. **Use this for arithmetic.**
`balanceFormatted`	Decimal FORGE for display only. Never compute with it.
`symbol`	`FORGE`, `tFORGE` or `dFORGE`
`blockNumber`	Block the balance was read at
`nonce`	Transaction count at that block

Status	Code	Meaning
`400`	`invalid_address`	Malformed address
`400`	`invalid_block`	Bad `block` value
`422`	`state_unavailable`	Historical block no longer available on your plan

```
const { balance } = await client.accounts.getBalance(address, { block:
"finalized" });
const enough = BigInt(balance) >= parseForge("5");
```

```
b = client.accounts.get_balance(address, block="finalized")
enough = int(b.balance) >= parse_forge("5")
```

5.4 `GET /accounts/balances`

Summary: Return native balances for up to 50 addresses in one request.

Required scope: `accounts:read`

Rate-limit cost: 1 request per 10 addresses, rounded up

Query parameters

Name	Type	Required	Description
`addresses`	string	**Required**	Comma-separated addresses, 1 to 50

Name	Type	Required	Description
`block`	string	*Optional*	Block selector. All balances are read at the **same** block, so the set is consistent.

Responses

`200 OK`

```
{
  "blockNumber": 4820141,
  "data": [
    { "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F", "balance":
"949999370000000000000" },
    { "address": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF", "balance":
"700000000000000000000" }
  ]
}
```

Status	Code	Meaning
`400`	`too_many_addresses`	More than 50 addresses
`400`	`invalid_address`	One or more invalid addresses; `details.addresses` lists them

```
curl "https://api.chainforge.example/v1/accounts/balances?
addresses=0x71C7656EC7ab88b098defB751B7401B5f6d8976F,0x2B5AD5c4795c026514f831
7c7a215E218DcCD6cF" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

This response is **not** paginated. If you need more than 50 balances, split them into batches.

5.5 `GET /accounts/{address}/nonce`

Summary: Return the nonce to use for an account's next transaction.

Required scope: `accounts:read`

Query parameters

Name	Type	Default	Description
`block`	string	`latest`	Supports `pending` in addition to the standard selectors

Description

Selector	Returned nonce
`latest` / `finalized`	Number of transactions **included** so far
`pending`	Included transactions **plus** transactions currently waiting in the mempool. Use this to send several transactions in a row.

Responses

`200 OK`

```
{ "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F", "nonce": 4,
"block": "pending" }
```

```
let nonce = await client.accounts.getNonce(address, { block: "pending" });
```

> **Note:** If you manage nonces locally for parallel sending (Developer Guide section 5.3), fetch the `pending` nonce once, then increment locally. Re-fetch after any submission error.

5.6 `GET /accounts/{address}/transactions`

Summary: List transactions sent from or to an account, newest first.

Required scope: `accounts:read`

Query parameters

Name	Type	Default	Description
`status`	string	all	Filter by status: `pending`, `included`, `finalized`, `failed`, `dropped`, `replaced`
`direction`	string	`any`	`sent`, `received` or `any`
`fromBlock`, `toBlock`	integer	none	Restrict to a block range (inclusive)
`limit`	integer	25	1 to 100 (section 3.3)
`cursor`	string	none	Pagination cursor

Responses

`200 OK`

```
{
  "data": [
    {
      "id": "txn_01JC9M6A1B",
      "hash": "0x4e1d0c7b2a9f...a9c3",
      "status": "finalized",
      "from": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
      "to": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
      "value": "5000000000000000000",
      "blockNumber": 4822095,
      "finalizedAt": "2026-10-18T10:07:41Z"
    }
  ],
  "nextCursor": null
}
```

Each item is a **Transaction** object (section 13). The list contains only transactions the platform has seen for this address; transactions that occurred before the address was registered are included once indexing completes (usually within 5 minutes for the first request).

Status	Code	Meaning
`400`	`invalid_cursor`	Cursor expired or used with different filters
`400`	`invalid_status`	Unknown `status` value

```
curl
"https://api.chainforge.example/v1/accounts/0x71C7656EC7ab88b098defB751B7401B
5f6d8976F/transactions?status=finalized&direction=received&limit=25" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
for await (const tx of client.accounts.listTransactions(address, { status:
"finalized" })) {
  console.log(tx.hash);
}
```

```
for tx in client.accounts.list_transactions(address, status="finalized"):
    print(tx.hash)
```

---

6. Transactions

A transaction is a signed instruction that changes ledger state. The REST API splits sending into two steps: **prepare** (the platform fills in the nonce, gas and fees for you) and **submit** (you send back the signed bytes). Signing always happens on your side: the API never receives a private key.

The standard flow

```
POST /transactions/prepare --> sign locally --> POST /transactions --> GET
/transactions/{id}  (or webhook)
```

6.1 `POST /transactions/prepare`

Summary: Build an unsigned transaction with the correct nonce, chain ID, gas limit and fee suggestions.

Required scope: `transactions:write`

Description

The response contains the exact bytes you need to sign. Fields you omit are filled in automatically. Calling this endpoint has no effect on the chain and costs nothing; you may call it as often as needed.

Request body

Field	Type	Required	Description
`from`	string<address>	**Required**	Sender address (the account that will sign)
`to`	string<address>	*Optional*	Recipient or contract. Omit to deploy a contract.
`value`	string<uint256>	*Optional*	Native amount in sparks. Default `"0"`.
`data`	string<hex>	*Optional*	Encoded contract call or creation code. Default `"0x"`.
`nonce`	integer	*Optional*	Override the nonce. Default: the sender's `pending` nonce.
`gasLimit`	string<uint256>	*Optional*	Override the gas limit. Default: estimate plus a 15 percent buffer.
`feeSpeed`	string	*Optional*	`slow`, `normal` (default) or `fast`
`maxFeePerGas`	string<uint256>	*Optional*	Explicit ceiling; overrides `feeSpeed`
`maxPriorityFeePerGas`	string<uint256>	*Optional*	Explicit tip; overrides `feeSpeed`

```
{
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "value": "2000000000000000000",
  "feeSpeed": "normal"
}
```

Responses

200 OK


```
{
  "chainId": 7701,
  "type": 2,
  "nonce": 1,
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "value": "2000000000000000000",
  "data": "0x",
  "gasLimit": "21000",
  "maxFeePerGas": "5000000000",
  "maxPriorityFeePerGas": "1000000000",
  "estimatedFee": "63000000000000",
  "maxFee": "105000000000000",
  "unsignedTransaction": "0x02f1830...",
  "signingHash":
"0x3a1f8c0d9b2e4f6a7c5d1e0b8a9f4c3d2e1b0a998877665544332211ffeeddcc",
  "expiresAt": "2026-10-18T09:40:12Z"
}
```


Field	Description
`chainId`	The network the transaction is valid on
`type`	Transaction format (2 = fee-market transaction)
`nonce`, `gasLimit`, `maxFeePerGas`, `maxPriorityFeePerGas`	The values that will be signed
`estimatedFee`	Expected fee in sparks at the current base fee
`maxFee`	Worst-case fee: `gasLimit x maxFeePerGas`
`unsignedTransaction`	The encoded transaction. Sign this.
`signingHash`	The hash that KMS and hardware signers sign

Field	Description
`expiresAt`	After this time the fee suggestions are stale and you should prepare again

Status	Code	Meaning
`400`	`validation_failed`	Bad field format
`422`	`insufficient_funds`	Balance below `value + maxFee`
`422`	`execution_reverted`	Estimation shows the call would revert; `details.revertReason` explains why
`422`	`wrong_network`	Address or data inconsistent with the network

```
curl -X POST https://api.chainforge.example/v1/transactions/prepare \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -d
'{"from":"0x71C7656EC7ab88b098defB751B7401B5f6d8976F","to":"0x2B5AD5c4795c026
514f8317c7a215E218DcCD6cF","value":"2000000000000000000"}'
```

```
const unsigned = await client.transactions.prepare({ from: wallet.address,
to, value: parseForge("2") });
const signed = await wallet.sign(unsigned);
```

6.2 `POST /transactions`

Summary: Submit a signed transaction to the network.

Required scope: `transactions:write`

Description

Validates the signature, nonce, balance and fee, then broadcasts the transaction. The call returns as soon as the transaction is **accepted into the mempool** (`202`), not when it is final. Track progress with `GET /transactions/{id}` , a webhook or a subscription.

Submitting the same signed bytes twice is safe: the second submission returns `409 already_known` with the existing transaction in `details.transaction` .

Headers

Name	Required	Description
`Idempotency-Key`	**Required**	Unique per logical operation (section 3.4)

Request body

Field	Type	Required	Description
`signedTransaction`	string<hex>	**Required**	The signed, encoded transaction, up to 128 KB
`metadata`	object	*Optional*	Up to 8 string key-value pairs stored with the transaction record, such as your `orderId`

```
{
  "signedTransaction": "0x02f8730183...",
  "metadata": { "orderId": "ord_8841" }
}
```

Responses

`202 Accepted`

```
{
  "id": "txn_01JC7X3M9Q",
  "hash":
"0x9f3c5d8a1b7e44c2a09d3f6b8e1c2d4a5b6c7d8e9f00112233445566778899aa",
  "status": "pending",
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "value": "2000000000000000000",
  "nonce": 1,
  "metadata": { "orderId": "ord_8841" },
  "submittedAt": "2026-10-18T09:30:12Z"
}
```

The `Location` header contains the URL of the new transaction resource.

Status	Code	Meaning
`400`	`invalid_signed_transaction`	Cannot decode or bad signature
`400`	`missing_idempotency_key`	Header absent
`409`	`already_known`	Identical transaction already in the mempool
`409`	`idempotency_conflict`	Key reused with a different body
`422`	`wrong_network`	Chain ID in the signed transaction does not match your key's network
`422`	`nonce_too_low`	Nonce already used
`422`	`nonce_gap`	An earlier nonce is missing; the transaction is held until the gap is filled
`422`	`insufficient_funds`	Balance below `value + maxFee`
`422`	`fee_too_low`	`maxFeePerGas` is below the current base fee
`422`	`gas_limit_too_low`	Limit cannot cover intrinsic gas

```
curl -X POST https://api.chainforge.example/v1/transactions \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -H "Idempotency-Key: 6f1d2c9a-3b7e-4d55-9a21-0e8c4b7a1f90" \
  -d '{"signedTransaction":"0x02f8730183..."}'
```

```
const tx = await client.transactions.submitSigned(signed, { idempotencyKey:
orderIdempotencyKey });
const receipt = await tx.waitForFinality();
```

```
tx = client.transactions.submit_signed(signed, idempotency_key=order_key)
receipt = tx.wait_for_finality()
```

6.3 `GET /transactions/{id}`

Summary: Retrieve a transaction by platform ID or by hash, including its current status and receipt data.

Required scope: `transactions:read`

Path parameters

Name	Type	Description
`id`	string	**Required.** A platform ID (`txn_...`) or a transaction hash (`0x...`)

Responses

`200 OK` : a finalized transfer

```
{
  "id": "txn_01JC7X3M9Q",
  "hash":
"0x9f3c5d8a1b7e44c2a09d3f6b8e1c2d4a5b6c7d8e9f00112233445566778899aa",
  "status": "finalized",
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "value": "2000000000000000000",
  "nonce": 1,
  "gasUsed": "21000",
  "effectiveGasPrice": "3000000000",
  "fee": "63000000000000",
  "blockNumber": 4820141,
  "blockHash":
"0xa1b2c3d4e5f60718293a4b5c6d7e8f90a1b2c3d4e5f60718293a4b5c6d7e8f90",
  "events": [],
  "submittedAt": "2026-10-18T09:30:12Z",
  "includedAt": "2026-10-18T09:30:14Z",
  "finalizedAt": "2026-10-18T09:30:16Z"
}
```

`200 OK` : a failed (reverted) contract call

```
{
  "id": "txn_01JC7Y9H4B",
  "status": "failed",
  "revertReason": "InsufficientPoints(available=250, required=10000)",
  "gasUsed": "38211",
  "fee": "1146330000000000",
  "blockNumber": 4821402
}
```

`200 OK`: a replaced transaction

```
{ "id": "txn_01JC7X3M9Q", "status": "replaced", "replacedBy":
"txn_01JC7ZB2WE" }
```

Status values

Status	Meaning	Terminal?
`pending`	Accepted, waiting in the mempool	No
`included`	In a block, not yet final	No
`finalized`	In a final block and succeeded	**Yes**
`failed`	Executed and reverted (fee consumed)	**Yes** (once finalized)
`dropped`	Left the mempool without inclusion (no fee)	**Yes**
`replaced`	Superseded by another transaction with the same nonce	**Yes**

Status	Code	Meaning
`404`	`transaction_not_found`	Unknown ID or hash. For a hash or ID you just created, retry for a few seconds (section 14.1).

```
curl https://api.chainforge.example/v1/transactions/txn_01JC7X3M9Q -H
"Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
const tx = await client.transactions.get("txn_01JC7X3M9Q");
if (tx.status === "failed") console.error(tx.revertReason);
```

6.4 `POST /transactions/{id}/speed-up`

Summary: Replace a pending transaction with an identical one that has a higher fee.

Required scope: `transactions:write`

Description

A stuck `pending` transaction can be replaced by a new one with the **same nonce** and a fee at least 10 percent higher. This endpoint prepares that replacement; **you sign and submit it** with `POST /transactions` as usual. Only `pending` transactions can be sped up.

Request body

Field	Type	Required	Description
`feeSpeed`	string	*Optional*	`normal` or `fast` (default `fast`)
`maxFeePerGas`	string<uint256>	*Optional*	Explicit ceiling; must be at least 110 percent of the original
`maxPriorityFeePerGas`	string<uint256>	*Optional*	Explicit tip; must be at least 110 percent of the original

Responses

`200 OK`: a PreparedTransaction (section 6.1) with the original nonce, recipient, value and data, and the increased fees.

Status	Code	Meaning
`404`	`transaction_not_found`	Unknown ID
`409`	`transaction_not_pending`	Already included, dropped or replaced
`422`	`fee_bump_too_low`	Fee increase is under 10 percent

```
const prepared = await client.transactions.speedUp("txn_01JC7X3M9Q", {
feeSpeed: "fast" });
const signed = await wallet.sign(prepared);
await client.transactions.submitSigned(signed, { idempotencyKey:
crypto.randomUUID() });
```

The SDK helper `transactions.speedUp(id, { signer })` performs all three steps.

6.5 `POST /transactions/{id}/cancel`

Summary: Prepare a zero-value self-transfer with the same nonce and a higher fee to displace a pending transaction.

Required scope: `transactions:write`

Description

Cancelling works by racing the original transaction: you sign and submit a harmless transaction that uses the same nonce with a higher fee. If it is included first, the original can never execute. **Cancellation is not guaranteed:** if the original is included before the replacement, it still executes. Check the final status.

Request body

Field	Type	Required	Description
`feeSpeed`	string	*Optional*	`normal` or `fast` (default `fast`)

Responses

`200 OK`: a PreparedTransaction with `to` equal to `from`, `value` of `"0"` and the original nonce.

Status	Code	Meaning
`409`	`transaction_not_pending`	The transaction is no longer pending
`404`	`transaction_not_found`	Unknown ID

```
curl -X POST
https://api.chainforge.example/v1/transactions/txn_01JC7X3M9Q/cancel \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -d '{"feeSpeed":"fast"}'
```

After signing and submitting the cancellation, the original transaction's status becomes `replaced` with `replacedBy` pointing to the cancellation.

---

7. Gas

Gas measures the computational work of a transaction, and the fee is `gasUsed x effectiveGasPrice` (see Developer Guide section 3.5). The gas endpoint helps you choose a sensible limit and fee settings before you sign.

7.1 `POST /gas/estimate`

Summary: Estimate the gas a transaction will use and return fee suggestions for three speeds.

Required scope: `transactions:read`

Description

The estimate **executes the call against current state without committing it**. As a result:

- If the call would revert, the endpoint returns `422 execution_reverted` with the revert reason, so you can pre-validate a call without spending gas.
- The estimate is accurate for the block it was run against, but state can change before your transaction is included. Add a buffer of 10 to 20 percent to `gasLimit` for contract calls. (`POST /transactions/prepare` adds a 15 percent buffer for you.)
- Fee suggestions are based on the current base fee and recent priority fees. They expire after about 30 seconds in practice, so estimate close to signing.

Request body

Field	Type	Required	Description
`from`	string<address>	**Required**	Sender. Needed because the result can depend on who calls.
`to`	string<address>	*Optional*	Recipient or contract. Omit to estimate a deployment.
`value`	string<uint256>	*Optional*	Sparks to transfer. Default `"0"`.
`data`	string<hex>	*Optional*	Encoded call data or creation code
`block`	string	*Optional*	State to estimate against (section 3.6). Default `latest`.

```
{
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "value": "5000000000000000000"
}
```

Responses

`200 OK`

```
{
  "gasLimit": "21000",
  "baseFee": "2000000000",
  "suggested": {
    "slow":   { "maxFeePerGas": "3000000000", "maxPriorityFeePerGas":
"500000000",  "estimatedWaitSeconds": 12 },
    "normal": { "maxFeePerGas": "5000000000", "maxPriorityFeePerGas":
"1000000000", "estimatedWaitSeconds": 4 },
    "fast":   { "maxFeePerGas": "8000000000", "maxPriorityFeePerGas":
"2500000000", "estimatedWaitSeconds": 2 }
  },
  "blockNumber": 4820141
}
```

Field	Type	Description
`gasLimit`	string<uint256>	Estimated gas the call will consume (before any buffer)
`baseFee`	string<uint256>	Current protocol base fee in sparks per gas
`suggested.<speed>.maxFeePerGas`	string<uint256>	Fee ceiling to set
`suggested.<speed>.maxPriorityFeePerGas`	string<uint256>	Tip to the validator
`suggested.<speed>.estimatedWaitSeconds`	integer	Typical time to inclusion at that fee
`blockNumber`	integer	Block the estimate was run against

Worked example: computing the fee from a response

For `normal` with `gasLimit` = 21,000 and the values above:

```
maxFee (worst case)  = 21,000 x 5,000,000,000 = 105,000,000,000,000 sparks =
0.000105 FORGE
effective price      = min(maxFeePerGas, baseFee + maxPriorityFeePerGas) =
min(5e9, 2e9 + 1e9) = 3e9
expected fee         = 21,000 x 3,000,000,000 = 63,000,000,000,000 sparks =
0.000063 FORGE
```

The sender's balance must cover `value + maxFee` at the time of submission.

Errors

Status	Code	Meaning
`400`	`validation_failed`	Bad field format
`422`	`execution_reverted`	The call would revert; `details.revertReason` explains why
`422`	`insufficient_funds`	The sender cannot cover `value`

Status	Code	Meaning
`422`	`state_unavailable`	Historical block unavailable on your plan

Example error

```
{
  "error": {
    "code": "execution_reverted",
    "message": "Estimation failed because the call would revert.",
    "details": { "revertReason": "ERC20: transfer amount exceeds balance" },
    "requestId": "req_3NQ8R2"
  }
}
```

Code samples

```
curl -X POST https://api.chainforge.example/v1/gas/estimate \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -d
'{"from":"0x71C7656EC7ab88b098defB751B7401B5f6d8976F","to":"0x2B5AD5c4795c026
514f8317c7a215E218DcCD6cF","value":"5000000000000000000"}'
```

```
const est = await client.gas.estimate({ from: wallet.address, to, value:
parseForge("5") });
const limit = (BigInt(est.gasLimit) * 115n) / 100n;   // add a 15 percent
buffer
```

```
est = client.gas.estimate(sender=wallet.address, to=to,
value=parse_forge("5"))
limit = int(est.gas_limit) * 115 // 100
```

---

8. Smart Contracts

These endpoints deploy contracts, register their interfaces (ABIs) so the platform can decode them, call and simulate functions, and query the events they emit. Writing to a contract is an ordinary transaction (chapter 6) whose `data` field carries the encoded call. The SDKs encode that data for you.

Terminology. The **ABI** is a JSON description of a contract's functions, events and errors. Registering a contract with its ABI lets you call functions by name and receive decoded results and decoded errors.

8.1 `POST /contracts`

Summary: Prepare a contract deployment transaction from compiled bytecode and constructor arguments.

Required scope: `contracts:write`

Description

Deployment is a transaction with no `to` address. This endpoint encodes the constructor arguments, estimates gas and returns a **PreparedTransaction** to sign and submit. When the deployment finalizes, the transaction receipt contains the new `contractAddress`. If you supply `registerOnDeploy: true` and an `abi`, the platform registers the contract automatically once it is final.

Headers

Name	Required	Description
`Idempotency-Key`	*Recommended*	Prevents duplicate preparation records

Request body

Field	Type	Required	Description
`from`	string<address>	**Required**	The deployer
`bytecode`	string<hex>	**Required**	Compiled creation bytecode, up to 256 KB
`abi`	array	**Required**	The contract ABI (JSON array)
`constructorArgs`	array	*Optional*	Constructor arguments, in order. Default `[]`.
`value`	string<uint256>	*Optional*	Sparks sent to a payable constructor

Field	Type	Required	Description
`name`	string	*Optional*	Display name for the registered contract, up to 64 characters
`registerOnDeploy`	boolean	*Optional*	Default `true`
`feeSpeed`	string	*Optional*	`slow`, `normal`, `fast`

```
{
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "name": "Acme Loyalty Points",
  "bytecode": "0x608060405234801561001057600080fd5b50...",
  "abi": [
    { "type": "function", "name": "balanceOf", "stateMutability": "view",
      "inputs": [{ "name": "account", "type": "address" }],
      "outputs": [{ "name": "", "type": "uint256" }] }
  ],
  "constructorArgs": [],
  "registerOnDeploy": true
}
```

Responses

`200 OK`

```
{
  "deploymentId": "dpl_01JCA2R7TG",
  "predictedAddress": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "transaction": {
    "chainId": 7701,
    "type": 2,
    "nonce": 0,
    "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
    "to": null,
    "value": "0",
    "gasLimit": "704269",
    "maxFeePerGas": "5000000000",
    "maxPriorityFeePerGas": "1000000000",
    "unsignedTransaction": "0x02f9...",
    "signingHash": "0x8c5d...e41f"
  }
}
```

Field	Description
`deploymentId`	Platform ID used to track the deployment
`predictedAddress`	The address the contract will have, derived from the deployer and nonce. It is valid only if you submit with this exact nonce.
`transaction`	A PreparedTransaction (section 6.1) to sign and submit via `POST /transactions`

After the transaction is `finalized`, `GET /transactions/{id}` includes `contractAddress`, and a `contract.deployed` webhook is sent if subscribed.

Errors

Status	Code	Meaning
`400`	`invalid_bytecode`	Not valid hex, or empty
`400`	`invalid_abi`	ABI is not a valid JSON array of items
`422`	`bytecode_too_large`	Above 256 KB

Status	Code	Meaning
`422`	`constructor_args_mismatch`	Arguments do not match the constructor in the ABI
`422`	`execution_reverted`	Constructor would revert

```
const deployment = await client.contracts.deploy({ abi, bytecode,
constructorArgs: [], signer: wallet });
const receipt = await deployment.waitForFinality();
console.log(receipt.contractAddress);
```

8.2 `POST /contracts/{address}/register`

Summary: Register an existing contract and its ABI so the platform can decode calls, results, errors and events.

Required scope: `contracts:write`

Description

Register contracts you did not deploy through the API, such as a token deployed from a template or a third-party contract. Once registered:

- `POST /contracts/{address}/call` and `simulate` accept function **names** and plain arguments.
- Transaction records include decoded `events` and decoded `revertReason`.
- `contract.event` webhooks and subscriptions can filter by event name and indexed arguments.

Registering again with a new ABI **replaces** the ABI (previous versions are retained for decoding historical events).

Path and body

Name	In	Type	Required	Description
`address`	path	string<address>	**Required**	Contract address; must have code on this network
`name`	body	string	*Optional*	Display name

Name	In	Type	Required	Description
`abi`	body	array	**Required**	The ABI
`startBlock`	body	integer	*Optional*	The block to begin indexing events from. Default: the deployment block, if known.

```
{ "name": "Acme Loyalty Points", "abi": [ ... ], "startBlock": 4821377 }
```

Responses

`201 Created` or **`200 OK`** (ABI replaced)

```
{
  "address": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "name": "Acme Loyalty Points",
  "abiVersion": 1,
  "functions": 5,
  "events": 2,
  "errors": 3,
  "indexing": { "status": "catching_up", "startBlock": 4821377 },
  "registeredAt": "2026-10-18T09:45:30Z"
}
```

Status	Code	Meaning
`404`	`contract_not_found`	No code at that address on this network
`400`	`invalid_abi`	Malformed ABI

```
curl -X POST
https://api.chainforge.example/v1/contracts/0x5FbDB2315678afecb367f032d93F642
f64180aa3/register \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -d @registration.json
```

8.3 `GET /contracts/{address}`

Summary: Retrieve a registered contract's metadata, ABI and indexing status.

Required scope: `contracts:read`

Query parameters

Name	Type	Default	Description
`includeAbi`	boolean	`false`	Include the full ABI in the response

Responses

`200 OK`

```
{
  "address": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "name": "Acme Loyalty Points",
  "abiVersion": 1,
  "deployedAt": "2026-10-18T09:44:10Z",
  "deploymentTransaction": "txn_01JCA2S1VQ",
  "verified": true,
  "indexing": { "status": "synced", "lastIndexedBlock": 4821402 },
  "abi": [ ]
}
```

Field	Description
`verified`	`true` once source code verification (`forge verify`) has matched the deployed bytecode
`indexing.status`	`catching_up` or `synced`
`indexing.lastIndexedBlock`	The newest block whose events are searchable

Status	Code	Meaning
`404`	`contract_not_registered`	The contract exists but has no registration in your project

8.4 `POST /contracts/{address}/call`

Summary: Execute a read-only (`view` or `pure`) function and return its decoded result. No transaction, no gas, no signature.

Required scope: `contracts:read`

Description

Use `call` to read data from a contract. If the function you name changes state, the call is rejected with `422 function_not_readonly`. Use a transaction instead (and `simulate` to preview it).

Because the contract is registered, you give a **function name** and plain **arguments**. If a function name is overloaded, give the full signature, for example `"balanceOf(address)"`.

Request body

Field	Type	Required	Description
`function`	string	**Required**	Function name or signature
`args`	array	*Optional*	Arguments in order. Large integers must be strings. Addresses must be valid.
`from`	string<address>	*Optional*	The caller address, for functions that depend on `msg.sender`
`block`	string	*Optional*	Block selector (section 3.6)

```
{ "function": "balanceOf", "args":
["0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF"], "block": "finalized" }
```

Responses

`200 OK`

```
{ "function": "balanceOf", "result": "250", "blockNumber": 4821390 }
```

For functions with several return values, `result` is an array in declaration order; for functions returning a struct, an object with named fields.

Status	Code	Meaning
`404`	`contract_not_registered`	Register the ABI first
`422`	`unknown_function`	No such function in the ABI
`422`	`function_not_readonly`	The function modifies state
`422`	`invalid_arguments`	Arguments do not match the function's inputs; `details` lists the expected types
`422`	`execution_reverted`	The function reverted

```
curl -X POST
https://api.chainforge.example/v1/contracts/0x5FbDB2315678afecb367f032d93F642
f64180aa3/call \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -d '{"function":"balanceOf","args":
["0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF"]}'
```

```
const points = client.contracts.at(contractAddress, abi);
const balance = await points.read.balanceOf([customerAddress]);
```

```
points = client.contracts.at(contract_address, abi)
balance = points.read.balance_of([customer_address])
```

8.5 `POST /contracts/{address}/simulate`

Summary: Run a state-changing function against current state **without committing it**, and report whether it would succeed.

Required scope: `contracts:read`

Rate-limit cost: 2 requests

Description

Simulation is the safest way to check a write before you pay for it. It reports whether the call would succeed, the gas it would use, the events it would emit and any revert reason. Nothing changes on the chain.

The result is accurate for the chosen block only; other transactions may change state before yours executes.

Request body

Field	Type	Required	Description
`function`	string	**Required**	Function name or signature
`args`	array	*Optional*	Arguments in order
`from`	string<address>	**Required**	The simulated caller
`value`	string<uint256>	*Optional*	Sparks sent with a payable function
`block`	string	*Optional*	Block to simulate against

```
{
  "function": "transfer",
  "args": ["0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF", "10000"],
  "from": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F"
}
```

Responses

`200 OK`: would succeed

```
{
  "success": true,
  "gasUsed": "51420",
  "result": true,
  "events": [
    { "name": "PointsTransferred", "args": { "from": "0x71C7...976F", "to":
"0x2B5A...D6cF", "amount": "10000" } }
  ],
  "blockNumber": 4821402
}
```

`200 OK`: would revert (this is **not** an HTTP error; check `success`)

```
{
  "success": false,
  "revertReason": "InsufficientPoints(available=250, required=10000)",
  "gasUsed": "29110",
  "blockNumber": 4821402
}
```

> **Note:** A simulation that reverts returns `200` with `success: false`, because the simulation itself worked. By contrast, `POST /gas/estimate` returns `422` for a reverting call, because it cannot produce an estimate.

```
const sim = await points.simulate.transfer([friend, 10_000n], { from:
wallet.address });
if (!sim.success) console.log("Would revert:", sim.revertReason);
```

8.6 `GET /contracts/{address}/events`

Summary: Query logs emitted by a registered contract, filtered by event name, block range and indexed arguments.

Required scope: `contracts:read`

Rate-limit cost: 2 requests when `limit` is above 50

Query parameters

Name	Type	Default	Description
`event`	string	all	Event name, or several separated by commas
`fromBlock`	string	deployment block	Start of range: a block number, `latest` or `finalized`
`toBlock`	string	`latest`	End of range (inclusive)
`finality`	string	`finalized`	`finalized` returns only final events; `included` also returns events from included blocks
`args.<name>`	string	none	Filter on an indexed argument, for example `args.to=0x2B5A...D6cF`

Name	Type	Default	Description
`limit`	integer	25	1 to 100
`cursor`	string	none	Pagination cursor
`order`	string	`asc`	`asc` (oldest first) or `desc`. Events default to **ascending** so you can replay in order.

> **Note:** Event lists are ordered oldest first by default, unlike most list endpoints, so that consumers can process them in on-chain order. Use `order=desc` for newest first.

Responses

`200 OK`

```
{
  "data": [
    {
      "address": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
      "event": "PointsIssued",
      "args": { "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF", "amount":
"100" },
      "blockNumber": 4821391,
      "blockHash": "0xd4e5...f601",
      "transactionHash": "0xb7c1...91ee",
      "logIndex": 0,
      "timestamp": "2026-10-18T09:50:02Z",
      "finalized": true
    }
  ],
  "nextCursor": null
}
```

Each item is an **EventLog** (section 13). The pair (`transactionHash`, `logIndex`) is globally unique and is the right key for de-duplication.

Status	Code	Meaning
`400`	`invalid_block_range`	`fromBlock` is after `toBlock`

Status	Code	Meaning
`400`	`block_range_too_large`	More than 100,000 blocks in one query; split the range
`404`	`contract_not_registered`	Register the contract first
`422`	`unknown_event`	Event name not in the ABI
`422`	`unknown_argument`	`args.<name>` is not an **indexed** argument of that event

```
curl
"https://api.chainforge.example/v1/contracts/0x5FbDB2315678afecb367f032d93F64
2f64180aa3/events?
event=PointsIssued&fromBlock=4821000&toBlock=latest&args.to=0x2B5AD5c4795c026
514f8317c7a215E218DcCD6cF&limit=100" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

```
for await (const ev of client.contracts.listEvents(contract, { events:
["PointsIssued"], fromBlock: 4821000 })) {
  console.log(ev.args.amount);
}
```

```
for ev in client.contracts.list_events(contract, events=["PointsIssued"],
from_block=4821000):
    print(ev.args["amount"])
```

---

9. Tokens and NFTs

Tokens are contracts that implement the CFT-20 (fungible) or CFT-721 (non-fungible) standards. These endpoints create tokens from audited templates and read token data without needing an ABI.

9.1 `POST /tokens`

Summary: Prepare the deployment of a CFT-20 or CFT-721 token from a platform template.

Required scope: `contracts:write`

Description

Templates deploy audited contract code with the features you choose. As with contract deployment, the endpoint returns a transaction to sign and submit. The token is automatically registered with its standard ABI when the deployment finalizes.

The feature flags you choose are **permanent** once deployed (except where the entry states otherwise).

Headers

Name	Required	Description
`Idempotency-Key`	**Required**	Prevents deploying two tokens by accident

Request body

Field	Type	Required	Description
`type`	string	**Required**	`cft20` or `cft721`
`name`	string	**Required**	Token name, 1 to 64 characters
`symbol`	string	**Required**	Ticker, 1 to 11 characters
`owner`	string<address>	**Required**	Account that will own and administer the token, and sign the deployment
`decimals`	integer	*Optional* (CFT-20)	0 to 18. Default 18. Ignored for CFT-721.
`initialSupply`	string<uint256>	*Optional* (CFT-20)	Units minted to `owner` at deployment, **in the smallest unit**
`maxSupply`	string<uint256>	*Optional*	Hard cap. Cannot be raised later. For CFT-721 it is a count of tokens.
`baseUri`	string	*Optional* (CFT-721)	Prefix for token metadata URIs, such as `ipfs://bafy.../`

Field	Type	Required	Description
`features.mintable`	boolean	*Optional*	The owner can create new units. Default `false`.
`features.burnable`	boolean	*Optional*	Holders can destroy their own units. Default `false`.
`features.pausable`	boolean	*Optional*	The owner can freeze transfers. Default `false`.
`features.allowList`	boolean	*Optional*	Restrict holders to an allow-list. Default `false`.

```
{
  "type": "cft20",
  "name": "Acme Dollar",
  "symbol": "ACMD",
  "decimals": 6,
  "initialSupply": "1000000000000",
  "maxSupply": "10000000000000",
  "owner": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "features": { "mintable": true, "burnable": true, "pausable": true }
}
```

In this example `initialSupply` of `1000000000000` equals 1,000,000.000000 tokens, because 6 decimals means one token is `1000000` units.

Responses

`200 OK`: a deployment object identical in shape to section 8.1, with the template's `predictedAddress` and a PreparedTransaction.

Status	Code	Meaning
`400`	`validation_failed`	Missing or malformed field
`422`	`invalid_decimals`	Outside 0 to 18
`422`	`initial_supply_exceeds_cap`	`initialSupply` is greater than `maxSupply`
`422`	`feature_unsupported`	Feature not available for this `type`

```
curl -X POST https://api.chainforge.example/v1/tokens \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -H "Idempotency-Key: 31d9c0aa-52f4-4f08-9d2e-18b6c5a1e7b3" \
  -d @token.json
```

9.2 `GET /tokens/{address}`

Summary: Retrieve a token's standard, name, symbol, decimals, supply and enabled features.

Required scope: `contracts:read`

Description

Works for any CFT-20 or CFT-721 contract on the network, **even one you have not registered**, by reading the standard functions directly.

Query parameters

Name	Type	Default	Description
`block`	string	`latest`	Block selector

Responses

`200 OK`: fungible token

```
{
  "address": "0x8464135c8F25Da09e49BC8782676a84730C318bC",
  "type": "cft20",
  "name": "Acme Dollar",
  "symbol": "ACMD",
  "decimals": 6,
  "totalSupply": "1000000000000",
  "maxSupply": "10000000000000",
  "owner": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
  "features": { "mintable": true, "burnable": true, "pausable": true,
"allowList": false },
  "paused": false,
  "blockNumber": 4822100
}
```

`200 OK` : non-fungible collection

```
{
  "address": "0x9A676e781A523b5d0C0e43731313A708CB607508",
  "type": "cft721",
  "name": "Acme Conference Tickets",
  "symbol": "ACT",
  "totalSupply": "42",
  "maxSupply": "500",
  "baseUri": "ipfs://bafybeih5kq.../",
  "blockNumber": 4822100
}
```

Status	Code	Meaning
`404`	`token_not_found`	No contract at the address
`422`	`not_a_token`	The contract does not implement CFT-20 or CFT-721

9.3 `GET /tokens/{address}/balances/{holder}`

Summary: Return how many units of a fungible token an address holds.

Required scope: `contracts:read`

Path and query parameters

Name	In	Type	Required	Description
`address`	path	string<address>	**Required**	Token contract
`holder`	path	string<address>	**Required**	Address whose balance to read
`block`	query	string	*Optional*	Block selector

Responses

`200 OK`

```
{
  "token": "0x8464135c8F25Da09e49BC8782676a84730C318bC",
  "holder": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
  "balance": "12500000",
  "balanceFormatted": "12.5",
  "symbol": "ACMD",
  "decimals": 6,
  "blockNumber": 4822100
}
```

For a CFT-721 collection this endpoint returns the **count** of tokens the holder owns in `balance`; use section 9.4 to list the individual tokens.

```
const b = await client.tokens.getBalance(token, holder);
console.log(b.balanceFormatted, b.symbol);
```

9.4 `GET /accounts/{address}/nfts`

Summary: List the non-fungible tokens an address owns.

Required scope: `accounts:read`

Query parameters

Name	Type	Default	Description
`collection`	string<address>	all	Restrict to one collection; comma-separate up to 10
`limit`	integer	25	1 to 100
`cursor`	string	none	Pagination cursor
`refreshMetadata`	boolean	`false`	Re-fetch metadata for the returned tokens (counts as 2 requests)

Responses

`200 OK`

```
{
  "data": [
    {
      "collection": "0x9A676e781A523b5d0C0e43731313A708CB607508",
      "tokenId": "42",
      "tokenUri": "ipfs://bafybeih5kq.../42.json",
      "name": "Acme Conference 2026 - Ticket #42",
      "description": "Admits one person to Acme Conference 2026.",
      "image":
"ipfs://bafybeigdyrzt5sfp7udm7hu76uh7y26nf3efuylqabf3oclgtqy55fbzdi",
      "attributes": [ { "trait_type": "Tier", "value": "General" } ],
      "metadataStatus": "ok"
    }
  ],
  "nextCursor": null
}
```

Field	Description
`tokenId`	Unique ID within the collection, as a string
`tokenUri`	The token's metadata location as stored on-chain
`name`, `description`, `image`, `attributes`	Taken from the metadata. Absent when `metadataStatus` is not `ok`.

Field	Description
`metadataStatus`	`ok`, `pending` (not fetched yet), `unreachable` (host did not respond) or `invalid` (not valid JSON or too large)

> **Note:** Metadata lives off-chain and the platform only caches it. A token's metadata can be unavailable even though the token itself is valid. Do not treat `unreachable` as proof that a token does not exist.

```
curl
"https://api.chainforge.example/v1/accounts/0x2B5AD5c4795c026514f8317c7a215E2
18DcCD6cF/nfts?collection=0x9A676e781A523b5d0C0e43731313A708CB607508" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

---

10. Webhooks

A **webhook** is an HTTPS endpoint on your server that ChainForge calls when something happens on-chain. The endpoints in this chapter manage those registrations, inspect delivery history and redeliver events. Sections 10.9 and 10.10 define the payloads and the signature scheme your server must handle.

For concepts and a walk-through, see Developer Guide chapter 8.

Delivery guarantees at a glance

Property	Behavior
Delivery	At least once. Duplicates are possible.
Order	Not guaranteed. Use `data.blockNumber` and `createdAt` to order.
Timeout	Your endpoint must respond with `2xx` within 10 seconds
Retries	Up to 7 attempts over about 32 hours (schedule in section 10.7)
Authenticity	Every request is signed (section 10.10)

Property	Behavior
Transport	HTTPS only for live webhooks; HTTP is allowed for devnet and `localhost` tunnels in test projects

10.1 `POST /webhooks`

Summary: Register a webhook endpoint and choose which events it receives.

Required scope: `webhooks:manage`

Description

When you create a webhook, the platform sends a verification request (a `webhook.ping` event) to the URL. The URL must respond with `2xx` within 10 seconds, or creation fails with `422 webhook_unreachable`. The response includes the signing `secret`, which is **shown only once**.

A project can have up to 20 active webhooks (Free plan: 3).

Request body

Field	Type	Required	Description
`url`	string	**Required**	Public HTTPS URL, up to 2,048 characters
`events`	array of string	**Required**	One or more event types (section 10.9). Use `["*"]` for all.
`description`	string	*Optional*	Free text, up to 256 characters
`filters`	object	*Optional*	Narrow which events are delivered (see below)
`enabled`	boolean	*Optional*	Default `true`
`headers`	object	*Optional*	Up to 5 custom headers added to every delivery, such as `{"X-Tenant": "acme"}`. Header names beginning with `X-ChainForge-` are reserved.

The `filters` object

Field	Type	Description
`addresses`	array<address>	Deliver only events involving these addresses (as sender, recipient or participant)
`contracts`	array<address>	For `contract.event`: only these contracts
`tokens`	array<address>	For `token.transfer`: only these tokens
`eventNames`	array of string	For `contract.event`: only these event names
`finality`	string	`finalized` (default) or `included`. Applies to `token.transfer`, `contract.event` and `account.received`.

A filter that does not apply to an event type is ignored for that type.

```
{
  "url": "https://shop.acme.example/hooks/chainforge",
  "description": "Order payment notifications",
  "events": ["transaction.finalized", "transaction.failed",
"token.transfer"],
  "filters": {
    "addresses": ["0x71C7656EC7ab88b098defB751B7401B5f6d8976F"],
    "tokens": ["0x8464135c8F25Da09e49BC8782676a84730C318bC"],
    "finality": "finalized"
  }
}
```

Responses

`201 Created`

```
{
  "id": "wh_01JC9K4TQZ",
  "url": "https://shop.acme.example/hooks/chainforge",
  "description": "Order payment notifications",
  "events": ["transaction.finalized", "transaction.failed",
"token.transfer"],
  "filters": { "addresses": ["0x71C7656EC7ab88b098defB751B7401B5f6d8976F"],
"tokens": ["0x8464135c8F25Da09e49BC8782676a84730C318bC"], "finality":
"finalized" },
  "enabled": true,
  "secret": "whsec_8f3b1c7d9e2a4b6c5d7e8f9a0b1c2d3e",
  "createdAt": "2026-10-18T10:05:00Z",
  "updatedAt": "2026-10-18T10:05:00Z"
}
```

> **Warning:** `secret` is returned only in this response (and in a rotate-secret response). Store it immediately in a secrets manager. All other responses omit it.

Status	Code	Meaning
`400`	`validation_failed`	Missing field or bad format
`400`	`invalid_event_type`	An entry in `events` is not a known type
`422`	`webhook_unreachable`	The verification ping did not return `2xx` within 10 seconds
`422`	`insecure_url`	Not HTTPS (live projects), or the host resolves to a private address
`422`	`webhook_limit_reached`	Plan limit on active webhooks

```
curl -X POST https://api.chainforge.example/v1/webhooks \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -d @webhook.json
```

```
const hook = await client.webhooks.create({
  url: "https://shop.acme.example/hooks/chainforge",
  events: ["transaction.finalized", "token.transfer"],
});
process.env.CHAINFORGE_WEBHOOK_SECRET = hook.secret; // store securely
instead of an env assignment
```

10.2 `GET /webhooks`

Summary: List the webhook endpoints in your project.

Required scope: `webhooks:manage`

Query parameters

Name	Type	Default	Description
`enabled`	boolean	all	Filter by enabled state
`event`	string	all	Only webhooks subscribed to this event type
`limit`	integer	25	1 to 100
`cursor`	string	none	Pagination cursor

Responses

`200 OK`

```
{
  "data": [
    {
      "id": "wh_01JC9K4TQZ",
      "url": "https://shop.acme.example/hooks/chainforge",
      "events": ["transaction.finalized", "transaction.failed",
"token.transfer"],
      "enabled": true,
      "health": { "status": "healthy", "lastSuccessAt": "2026-10-
18T10:07:41Z", "consecutiveFailures": 0 },
      "createdAt": "2026-10-18T10:05:00Z"
    }
  ],
  "nextCursor": null
}
```

The `secret` is never included in list or read responses.

10.3 `GET /webhooks/{id}`

Summary: Retrieve one webhook endpoint, including its health.

Required scope: `webhooks:manage`

Responses

`200 OK`: a **Webhook** object (section 13) with the same fields as in 10.1 minus `secret`, plus:

Field	Type	Description
`health.status`	string	`healthy`, `degraded` (recent failures) or `disabled`
`health.lastSuccessAt`	string<datetime>	Last successful delivery
`health.lastFailureAt`	string<datetime>	Last failed delivery
`health.consecutiveFailures`	integer	Failures since the last success
`disabledReason`	string	Present when disabled by the platform, for example `repeated_failures`

Status	Code	Meaning
`404`	`webhook_not_found`	Unknown ID or belongs to another project

10.4 `PATCH /webhooks/{id}`

Summary: Update a webhook's URL, events, filters, headers or enabled state.

Required scope: `webhooks:manage`

Description

Send only the fields you want to change. `events`, `filters` and `headers`, when sent, **replace** the existing value entirely. Changing `url` triggers a new verification ping. Setting `enabled` to `true` on a platform-disabled webhook re-enables it and resets `consecutiveFailures`.

Request body

Any subset of `url`, `description`, `events`, `filters`, `headers`, `enabled` (definitions as in 10.1).

```
{ "events": ["transaction.finalized"], "enabled": true }
```

Responses

`200 OK`: the updated Webhook object (without `secret`).

Status	Code	Meaning
`404`	`webhook_not_found`	Unknown ID
`422`	`webhook_unreachable`	New URL failed the ping

```
curl -X PATCH https://api.chainforge.example/v1/webhooks/wh_01JC9K4TQZ \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY" -H "Content-Type:
application/json" \
  -d '{"enabled": false}'
```

10.5 `DELETE /webhooks/{id}`

Summary: Delete a webhook endpoint.

Required scope: `webhooks:manage`

Description

Deletion is immediate and permanent. Deliveries in progress are abandoned, and delivery history is retained for 30 days for audit but can no longer be redelivered. To pause delivery temporarily, use `PATCH` with `enabled: false` instead.

Responses

`204 No Content`

Status	Code	Meaning
`404`	`webhook_not_found`	Unknown ID (deleting twice is therefore an error)

```
curl -X DELETE https://api.chainforge.example/v1/webhooks/wh_01JC9K4TQZ -H
"Authorization: Bearer $CHAINFORGE_API_KEY"
```

10.6 `POST /webhooks/{id}/rotate-secret`

Summary: Issue a new signing secret, with a 24-hour overlap during which both secrets are valid.

Required scope: `webhooks:manage`

Description

Rotate when you suspect exposure, when a team member with access leaves, or on a schedule. During the overlap, every delivery is signed with **both** secrets so your server can switch without dropping events (see section 10.10).

A safe rotation procedure

1. Call this endpoint and store the new secret.
2. Deploy a server version that accepts a signature made with **either** secret.
3. Confirm in the delivery log that requests verify with the new secret.

4. Remove the old secret from your server. The platform stops signing with it 24 hours after rotation.

Request body

None.

Responses

`200 OK`

```
{
  "id": "wh_01JC9K4TQZ",
  "secret": "whsec_a1b2c3d4e5f60718293a4b5c6d7e8f90",
  "previousSecretExpiresAt": "2026-10-19T11:00:00Z"
}
```

Status	Code	Meaning
`404`	`webhook_not_found`	Unknown ID
`409`	`rotation_in_progress`	A previous rotation's overlap period has not ended

10.7 `GET /webhooks/{id}/deliveries`

Summary: List delivery attempts for a webhook, with request and response details.

Required scope: `webhooks:manage`

Query parameters

Name	Type	Default	Description
`status`	string	all	`succeeded`, `failed` or `pending` (waiting for a retry)
`eventType`	string	all	Filter by event type
`since`	string<datetime>	30 days ago	Earliest attempt time. Delivery history is kept for 30 days.
`limit`	integer	25	1 to 100

Name	Type	Default	Description
`cursor`	string	none	Pagination cursor

Responses

`200 OK`


```
{
  "data": [
    {
      "eventId": "evt_01JC9M7N2X",
      "eventType": "transaction.finalized",
      "status": "succeeded",
      "attempts": [
        {
          "attempt": 1,
          "sentAt": "2026-10-18T10:07:42Z",
          "responseStatus": 200,
          "durationMs": 184,
          "error": null
        }
      ],
      "nextAttemptAt": null
    },
    {
      "eventId": "evt_01JC9M8B4Y",
      "eventType": "token.transfer",
      "status": "pending",
      "attempts": [
        { "attempt": 1, "sentAt": "2026-10-18T10:09:10Z", "responseStatus":
503, "durationMs": 9210, "error": null },
        { "attempt": 2, "sentAt": "2026-10-18T10:09:40Z", "responseStatus":
null, "durationMs": 10000, "error": "timeout" }
      ],
      "nextAttemptAt": "2026-10-18T10:14:40Z"
    }
  ],
  "nextCursor": null
}
```

Field	Description
`eventId`	The ID of the event; also the key for `redeliver`
`status`	`succeeded`, `failed` (all attempts used) or `pending`
`attempts[].responseStatus`	HTTP status your server returned, or `null` if none
`attempts[].error`	`timeout`, `connection_refused`, `tls_error`, `dns_error` or `null`
`nextAttemptAt`	When the next retry is scheduled, if pending

Retry schedule

Attempt	Delay after previous failure
1	Immediate
2	30 seconds
3	5 minutes
4	30 minutes
5	2 hours
6	6 hours
7	24 hours

After the seventh failed attempt the delivery is `failed`. If a webhook fails continuously for three days the platform disables it and emails project administrators.

10.8 `POST /webhooks/{id}/deliveries/{eventId}/redeliver`

Summary: Send a previous event again to the webhook's current URL.

Required scope: `webhooks:manage`

Description

Use this to recover events missed during an outage of your server. The redelivered request has the **same** `id` as the original event, so a correctly idempotent handler ignores it if the event was already processed. It is signed with the secret(s) valid **now**, with a fresh timestamp. Events older than 30 days cannot be redelivered.

Responses

`202 Accepted`

```
{ "eventId": "evt_01JC9M7N2X", "status": "pending", "nextAttemptAt": "2026-10-18T10:20:01Z" }
```

Status	Code	Meaning
`404`	`delivery_not_found`	Unknown event or beyond the 30-day window
`409`	`webhook_disabled`	Re-enable the webhook first
`429`	`redelivery_rate_limited`	More than 60 redeliveries per minute

```
curl -X POST https://api.chainforge.example/v1/webhooks/wh_01JC9K4TQZ/deliveries/evt_01JC9M7N2X/redeliver \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

10.9 Webhook Event Reference

Every delivery is an HTTP `POST` with a JSON body in this envelope:

Field	Type	Description
`id`	string	Unique event ID (`evt_...`). **Use it for de-duplication.**
`type`	string	Event type from the table below
`apiVersion`	string	API version that shaped the payload, such as `v1`
`createdAt`	string<datetime>	When the platform created the event

Field	Type	Description
`network`	string	`mainnet`, `ember` or `devnet`
`data`	object	Event-specific payload (below)

Event types

Type	Sent when	`data` is
`webhook.ping`	A webhook is created or its URL changes, or a test is requested	`{ "message": "..." }`
`transaction.pending`	A transaction is accepted into the mempool	Transaction
`transaction.included`	A transaction is placed in a block	Transaction
`transaction.finalized`	A transaction reaches finality and succeeded	Transaction
`transaction.failed`	A transaction was included but reverted	Transaction (with `revertReason`)
`transaction.dropped`	A transaction left the mempool without inclusion	Transaction
`account.received`	A registered address receives native FORGE	TransferInfo
`token.transfer`	A watched token moves (including mints and burns)	TokenTransfer
`contract.event`	A registered contract emits a matching log	EventLog
`contract.deployed`	A contract deployment finalizes	ContractDeployed

Payload examples

`transaction.finalized`

```
{
  "id": "evt_01JC9M7N2X",
  "type": "transaction.finalized",
  "apiVersion": "v1",
  "createdAt": "2026-10-18T10:07:41Z",
  "network": "ember",
  "data": {
    "id": "txn_01JC9M6A1B",
    "hash":
"0x4e1d0c7b2a9f3e5d6c8b7a1f0e2d3c4b5a69788796a5b4c3d2e1f0011223a9c3",
    "status": "finalized",
    "from": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
    "to": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
    "value": "5000000000000000000",
    "blockNumber": 4822095,
    "metadata": { "orderId": "ord_8841" }
  }
}
```

`transaction.failed`

```
{
  "id": "evt_01JC9N2D5Q",
  "type": "transaction.failed",
  "apiVersion": "v1",
  "createdAt": "2026-10-18T10:15:03Z",
  "network": "ember",
  "data": {
    "id": "txn_01JC9N1Z8F",
    "hash": "0x77aa5f01...c01e",
    "status": "failed",
    "revertReason": "InsufficientPoints(available=250, required=10000)",
    "gasUsed": "38211",
    "blockNumber": 4822210
  }
}
```

`token.transfer`

```
{
  "id": "evt_01JC9P4K7R",
  "type": "token.transfer",
  "apiVersion": "v1",
  "createdAt": "2026-10-18T10:21:29Z",
  "network": "ember",
  "data": {
    "token": "0x8464135c8F25Da09e49BC8782676a84730C318bC",
    "from": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
    "to": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
    "value": "12500000",
    "transactionHash":
"0xb7c1d2e3f4a5968778695a4b3c2d1e0f00112233445566778899aabbccdd91ee",
    "logIndex": 2,
    "blockNumber": 4822310,
    "finalized": true
  }
}
```

A **mint** has `from` equal to the zero address (`0x0000000000000000000000000000000000000000`); a **burn** has `to` equal to the zero address. For CFT-721 transfers, `data` has `tokenId` instead of `value`.

`contract.event`

```
{
  "id": "evt_01JC9Q8M1S",
  "type": "contract.event",
  "apiVersion": "v1",
  "createdAt": "2026-10-18T10:30:55Z",
  "network": "ember",
  "data": {
    "address": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
    "event": "PointsIssued",
    "args": { "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF", "amount":
"100" },
    "blockNumber": 4822401,
    "transactionHash": "0xc9d8...e7f6",
    "logIndex": 0,
    "finalized": true
  }
}
```

`contract.deployed`

```
{
  "id": "evt_01JC9R1V3T",
  "type": "contract.deployed",
  "apiVersion": "v1",
  "createdAt": "2026-10-18T10:35:10Z",
  "network": "ember",
  "data": {
    "deploymentId": "dpl_01JCA2R7TG",
    "contractAddress": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
    "deployer": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
    "transactionHash": "0x77aa...c01e",
    "blockNumber": 4821377
  }
}
```

`account.received`

```
{
  "id": "evt_01JC9S6B9U",
  "type": "account.received",
  "apiVersion": "v1",
  "createdAt": "2026-10-18T10:40:02Z",
  "network": "ember",
  "data": {
    "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
    "from": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF",
    "value": "1000000000000000000",
    "transactionHash": "0xd1e2...f3a4",
    "blockNumber": 4822500,
    "finalized": true
  }
}
```

Handling advice

- Return `200` quickly, and process in the background.
- Never depend on arrival order; compare `blockNumber`.
- Use `id` for de-duplication, and (`transactionHash`, `logIndex`) for on-chain events.
- For irreversible actions, confirm with `GET /transactions/{id}` before acting.

10.10 Signature Verification

Every delivery carries these headers:

Header	Value
`X-ChainForge-Timestamp`	Unix time in seconds when the request was signed
`X-ChainForge-Signature`	`v1=<hex>`: an HMAC-SHA256 signature. During a secret rotation overlap it contains two comma-separated values: `v1=<hex_new>,v1=<hex_old>`.
`X-ChainForge-Event-Id`	The event ID, equal to the body's `id`
`X-ChainForge-Delivery-Attempt`	Attempt number, starting at 1
`User-Agent`	`ChainForge-Webhooks/1.0`
`Content-Type`	`application/json`

Algorithm

```
signed_payload = timestamp + "." + raw_request_body
signature      = HMAC_SHA256(secret, signed_payload)   -- hex encoded
header value   = "v1=" + signature
```

Verification procedure

1. Read the **raw** body bytes. Do not parse and re-serialize them.
2. Read `X-ChainForge-Timestamp`; reject the request if it differs from your clock by more than 5 minutes.
3. Compute the HMAC-SHA256 of `timestamp + "." + rawBody` with your secret.
4. Compare with each `v1=` value in the header using a **constant-time** comparison. Accept if any matches.
5. Parse the JSON and process the event.

Worked test vector

Use this to test your implementation.

Item	Value
Secret	`whsec_test_secret`
Timestamp	`1795000000`
Body	`{"id":"evt_test","type":"webhook.ping"}`
Signed payload	`1795000000.` `{"id":"evt_test","type":"webhook.ping"}`

Your HMAC-SHA256 hex digest of the signed payload with that secret must equal the value after `v1=` in the header for this sample request. (Compute it with your own implementation and compare it with the Console's **Test signature** tool, which accepts these same inputs.)

Reference implementations

```
import crypto from "node:crypto";

export function verify(rawBody, headers, secrets) {
  const ts = headers["x-chainforge-timestamp"];
  if (!ts || Math.abs(Date.now() / 1000 - Number(ts)) > 300) return false;

  const provided = (headers["x-chainforge-signature"] || "")
    .split(",").map(s => s.trim().replace(/^v1=/, ""));

  return secrets.some(secret => {
    const expected = crypto.createHmac("sha256",
secret).update(`${ts}.${rawBody}`).digest("hex");
    return provided.some(p =>
      p.length === expected.length &&
      crypto.timingSafeEqual(Buffer.from(p), Buffer.from(expected)));
  });
}
```

```
import hmac, hashlib, time

def verify(raw_body: bytes, headers: dict, secrets: list[str]) -> bool:
    ts = headers.get("X-ChainForge-Timestamp", "")
    if not ts.isdigit() or abs(time.time() - int(ts)) > 300:
        return False
    provided = [p.strip().removeprefix("v1=") for p in headers.get("X-
ChainForge-Signature", "").split(",")]
    for secret in secrets:
        expected = hmac.new(secret.encode(), ts.encode() + b"." + raw_body,
hashlib.sha256).hexdigest()
        if any(hmac.compare_digest(p, expected) for p in provided):
            return True
    return False
```

The SDKs wrap this as `client.webhooks.constructEvent(rawBody, headers, secret)`.

Common verification failures

Symptom	Cause
Signatures never match	The body was parsed and re-serialized before verification, or middleware changed the bytes
Intermittent failures	Server clock skew above 5 minutes; synchronize time
Fails only after rotation	Server still has only the old secret; accept both during the overlap
Works locally, fails in production	A proxy modifies the body or strips headers

---

11. Usage

11.1 `GET /usage`

Summary: Return request, error and quota usage for your project over a time period.

Required scope: `admin`

Description

Use this endpoint to feed your own dashboards and alerts. Figures update about once a minute. Usage is reported per project and, optionally, per API key or endpoint group.

Query parameters

Name	Type	Default	Description
`from`	string<datetime>	24 hours ago	Start of the period (inclusive)
`to`	string<datetime>	now	End of the period (exclusive)
`granularity`	string	`hour`	`minute` (max 6 hours of range), `hour` (max 31 days), `day` (max 366 days)
`groupBy`	string	none	`key`, `endpoint` or `status`
`keyId`	string	all	Restrict to one API key (its ID, not its secret)

Responses

`200 OK`

```
{
  "from": "2026-10-18T00:00:00Z",
  "to": "2026-10-18T06:00:00Z",
  "granularity": "hour",
  "plan": "free",
  "limits": { "requestsPerSecond": 10, "requestsPerDay": 100000,
"webSocketSubscriptions": 5 },
  "totals": { "requests": 18420, "errors": 311, "rateLimited": 12,
"errorRate": 0.0169 },
  "series": [
    { "start": "2026-10-18T00:00:00Z", "requests": 2100, "errors": 31,
"rateLimited": 0, "p95LatencyMs": 142 },
    { "start": "2026-10-18T01:00:00Z", "requests": 1850, "errors": 22,
"rateLimited": 0, "p95LatencyMs": 138 }
  ],
  "quota": { "dailyUsed": 18420, "dailyRemaining": 81580, "resetsAt": "2026-
10-19T00:00:00Z" }
}
```

Field	Description
`plan`	`free`, `growth` or `enterprise`
`limits`	Plan limits (section 3.5)
`totals.requests`	All requests in the period
`totals.errors`	Responses with status 400 or above
`totals.rateLimited`	Requests answered `429`
`totals.errorRate`	`errors / requests`, between 0 and 1
`series[]`	One entry per time bucket
`series[].p95LatencyMs`	95th-percentile response time in milliseconds
`quota`	Daily quota consumption and reset time (00:00 UTC)

With `groupBy`, each series entry gains a `group` field (for example, the key ID or endpoint such as `POST /transactions`).

Status	Code	Meaning
`400`	`invalid_time_range`	`from` is after `to`, or the range exceeds the granularity's limit
`403`	`insufficient_scope`	Credential lacks `admin`

```
curl "https://api.chainforge.example/v1/usage?from=2026-10-
18T00:00:00Z&to=2026-10-18T06:00:00Z&granularity=hour&groupBy=endpoint" \
  -H "Authorization: Bearer $CHAINFORGE_ADMIN_KEY"
```

```
const usage = await client.usage.get({ granularity: "hour", groupBy:
"endpoint" });
if (usage.totals.errorRate > 0.02) alertOnCall("API error rate above 2
percent");
```

---

12. Real-Time Subscriptions (WebSocket)

WebSocket subscriptions stream chain events to a long-running process with lower latency than polling and without exposing a public endpoint (unlike webhooks). Use them for dashboards, bots and services that stay connected. For serverless or multi-instance backends, webhooks are usually a better fit (Developer Guide section 8.1 compares the options).

12.1 Connection and Authentication

Item	Value
Mainnet URL	`wss://ws.chainforge.example/v1`
Ember URL	`wss://ws.ember.chainforge.example/v1`
Protocol	JSON text frames over WebSocket (RFC 6455), TLS required
Authentication	An API key or OAuth access token with `accounts:read` and `contracts:read`

Item	Value
Idle timeout	Connections idle for 60 seconds without any frame are closed. Send a `ping` at least every 30 seconds.
Maximum message size	64 KB
Subscription limits	Per plan (section 3.5): 5, 50 or custom concurrent subscriptions

Authenticating

Send the credential in the `Authorization` header of the upgrade request where your client allows it:

```
GET /v1 HTTP/1.1
Host: ws.ember.chainforge.example
Upgrade: websocket
Connection: Upgrade
Authorization: Bearer cfk_test_8hG2kLm9PqRsT4vWxYz1AbCdEfGhJkLm
```

Browsers cannot set headers on WebSocket connections. Instead, connect first and send an `auth` message within 10 seconds, otherwise the server closes the connection with code `4001`:

```
{ "op": "auth", "token": "cfk_test_8hG2kLm9PqRsT4vWxYz1AbCdEfGhJkLm" }
```

> **Warning:** Do not place a secret API key in browser code. For browser clients, mint a short-lived OAuth token on your server with only the read scopes needed and pass that token.

Server reply

```
{ "op": "auth.ok", "connectionId": "conn_01JCB3M5XK", "network": "ember",
"limits": { "subscriptions": 5 } }
```

Close codes

Code	Meaning	Reconnect?
`1000`	Normal closure	Optional
`1001`	Server going away (deployment)	Yes, immediately
`1008`	Policy violation (for example, oversize message)	No, fix the client
`4001`	Authentication timeout	Yes, after sending `auth` promptly
`4003`	Authentication failed or insufficient scope	No
`4008`	Rate limited (too many messages)	Yes, with backoff
`4009`	Subscription limit exceeded	No, unsubscribe first
`4010`	Idle timeout	Yes

12.2 Subscription Message Format

All messages are JSON objects with an `op` field.

Client messages

`op`	Purpose	Fields
`auth`	Authenticate (when headers are unavailable)	`token`
`subscribe`	Start a subscription	`id`, `type`, type-specific filters, `finality`, `fromBlock`
`unsubscribe`	Stop a subscription	`id`
`ping`	Keep the connection alive	none

Server messages

`op`	Purpose	Fields
`auth.ok`	Authentication succeeded	`connectionId`, `network`, `limits`
`subscribed`	A subscription is active	`id`, `type`, `startBlock`
`unsubscribed`	A subscription ended	`id`
`event`	A matching event	`subscription`, `data`, `cursor`
`error`	A request failed	`id` (if related), `code`, `message`
`pong`	Reply to `ping`	none
`notice`	Informational (for example, an upcoming maintenance window)	`message`

The `subscribe` message

Field	Type	Required	Description
`id`	string	**Required**	Your own identifier for this subscription, unique on the connection, up to 64 characters
`type`	string	**Required**	See section 12.3
`finality`	string	*Optional*	`finalized` (default) or `included`
`fromBlock`	integer	*Optional*	Begin from this block (replaying history up to 10,000 blocks back) instead of from now
`resumeFrom`	string	*Optional*	The `cursor` of the last event you processed (section 12.4). Takes precedence over `fromBlock`.
(type-specific filters)			See section 12.3

Example exchange

```
// client -> server
{ "op": "subscribe", "id": "points-issued", "type": "contract.event",
  "contract": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "events": ["PointsIssued"], "finality": "finalized" }

// server -> client
{ "op": "subscribed", "id": "points-issued", "type": "contract.event",
"startBlock": 4822500 }

// server -> client, later
{
  "op": "event",
  "subscription": "points-issued",
  "cursor": "4822511:0:3",
  "data": {
    "address": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
    "event": "PointsIssued",
    "args": { "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF", "amount":
"100" },
    "blockNumber": 4822511,
    "transactionHash": "0xe5f6...a7b8",
    "logIndex": 0,
    "finalized": true
  }
}
```

Error example

```
{ "op": "error", "id": "points-issued", "code": "unknown_event", "message":
"Event 'PointsIssuedX' is not in the contract ABI." }
```

12.3 Subscription Types

`type`	Fires when	Filters
`block.new`	A block is produced (`included`) or finalized (`finalized`)	none

`type`	Fires when	Filters
`transaction.status`	A watched transaction changes status	`transactionId` or `hash` (one required)
`account.activity`	A watched address sends or receives native FORGE	`address` (required), `direction`: `sent`, `received` or `any`
`contract.event`	A registered contract emits a matching log	`contract` (required), `events` (array), `args` (object of indexed argument filters)
`token.transfer`	A watched token moves (including mints and burns)	`token` (required), `from`, `to`

`block.new`

```
{ "op": "subscribe", "id": "heads", "type": "block.new", "finality":
"finalized" }
```

Event `data`: a compact block object with `number`, `hash`, `timestamp`, `transactionCount` and `finalized`.

`transaction.status`

```
{ "op": "subscribe", "id": "watch-9f3c", "type": "transaction.status",
"transactionId": "txn_01JC7X3M9Q" }
```

Event `data`: `{ "id": "txn_01JC7X3M9Q", "status": "finalized", "blockNumber": 4820141 }`. The subscription ends automatically after a terminal status (`finalized`, `failed`, `dropped`, `replaced`) and the server sends `unsubscribed`.

`account.activity`

```
{ "op": "subscribe", "id": "alice", "type": "account.activity",
  "address": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F", "direction":
"received" }
```

Event `data`: `address`, `counterparty`, `direction`, `value`, `transactionHash`, `blockNumber`, `finalized`.

`contract.event`

```
{ "op": "subscribe", "id": "to-bob", "type": "contract.event",
  "contract": "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  "events": ["PointsTransferred"],
  "args": { "to": "0x2B5AD5c4795c026514f8317c7a215E218DcCD6cF" } }
```

Event `data`: an **EventLog** (section 13). Filters in `args` work only on **indexed** event arguments.

`token.transfer`

```
{ "op": "subscribe", "id": "acmd", "type": "token.transfer",
  "token": "0x8464135c8F25Da09e49BC8782676a84730C318bC",
  "to": "0x71C7656EC7ab88b098defB751B7401B5f6d8976F" }
```

Event `data`: `token`, `from`, `to`, `value` (or `tokenId` for CFT-721), `transactionHash`, `logIndex`, `blockNumber`, `finalized`.

`finality` semantics

Value	Behavior	Use for
`finalized`	Each event is delivered once, after finality. Never retracted.	Money, inventory, accounting
`included`	Delivered as soon as the event is in a block (about 2 seconds sooner). In rare cases of reordering an event is followed by a `reorg` notice.	Live displays

With `included`, a retraction arrives as `{ "op": "event", "subscription": "...", "data": { ... }, "removed": true }` for an event that is no longer in the canonical chain. Consumers using `included` must handle `removed: true`.

12.4 Reconnection and Resume

Connections drop for many reasons: deployments, network changes, idle timeouts. A robust client reconnects and **resumes without losing or duplicating events**.

Cursors

Every `event` message includes a `cursor`, an opaque string that encodes the position of the event in the stream. Persist the cursor of the **last event you fully processed**, per subscription.

Resume procedure

1. Reconnect with exponential backoff and jitter (1 s, 2 s, 4 s, up to 30 s).
2. Authenticate.
3. Re-send each `subscribe` with `resumeFrom` set to the stored cursor.
4. The server replays everything after that cursor (up to 10,000 blocks of history) and then continues live.
5. De-duplicate using (`transactionHash`, `logIndex`), because the replay may include an event you processed just before the drop.

```
{ "op": "subscribe", "id": "points-issued", "type": "contract.event",
  "contract": "0x5FbDB2315678afecb367f032d93F642f64180aa3", "events":
["PointsIssued"],
  "resumeFrom": "4822511:0:3" }
```

If the gap is larger than the 10,000-block replay window, the server replies with `error` code `resume_window_exceeded`. In that case backfill the gap using `GET /contracts/{address}/events` (section 8.6) and then subscribe from now.

Heartbeats

```
{ "op": "ping" }   // client, every 20 to 30 seconds
{ "op": "pong" }   // server reply
```

If no `pong` arrives within 10 seconds, treat the connection as dead and reconnect.

SDK behavior

The official SDKs implement authentication, heartbeats, reconnection with backoff, cursor persistence in memory and automatic resume. To survive process restarts, store `event.cursor` in your database and pass it as `resumeFrom` when creating the subscription.

```
const sub = client.subscribe({
  type: "contract.event",
  contract: "0x5FbDB2315678afecb367f032d93F642f64180aa3",
  events: ["PointsIssued"],
  finality: "finalized",
  resumeFrom: await db.getCursor("points-issued"),
});
sub.on("event", async (e) => {
  await applyEvent(e.data);
  await db.saveCursor("points-issued", e.cursor);   // after the event is
fully processed
});
```

---

13. Models (Schemas)

This chapter defines every object that appears in requests and responses. Field tables list **Type**, whether the field is always present (**Req**) and a description. Unless noted, fields marked *no* are omitted when they have no value.

13.1 Common Types

Type	Definition
`address`	42-character hex string beginning `0x` (responses use checksum casing)
`hash`	66-character hex string beginning `0x`
`hex`	`0x`-prefixed hexadecimal bytes of even length
`uint256`	Non-negative integer as a decimal string, up to 78 digits
`datetime`	ISO 8601 UTC, such as `2026-10-18T09:30:12Z`
`BlockSelector`	`latest`, `finalized`, `pending` (where supported) or a block number

13.2 Account

Field	Type	Req	Description
`address`	address	yes	The account address
`type`	string	yes	`eoa` or `contract`
`balance`	uint256	yes	Native balance in sparks
`nonce`	integer	yes	Transaction count (EOA) or contracts created (contract)
`registered`	boolean	yes	Whether registered with your project
`label`	string	no	Your label (registered accounts only)
`metadata`	object	no	Your key-value pairs (registered accounts only)
`codeHash`	hash	no	Contracts only
`codeSize`	integer	no	Contracts only, in bytes
`blockNumber`	integer	yes	Block at which state was read

13.3 Balance

Field	Type	Req	Description
`address`	address	yes	The account
`balance`	uint256	yes	Sparks
`balanceFormatted`	string	yes	Display decimal in FORGE. Not for arithmetic.
`symbol`	string	yes	`FORGE`, `tFORGE` or `dFORGE`
`blockNumber`	integer	yes	Block at which state was read
`nonce`	integer	yes	Transaction count

13.4 Block

Field	Type	Req	Description
`number`	integer	yes	Block height
`hash`	hash	yes	Block hash
`parentHash`	hash	yes	Previous block's hash
`timestamp`	datetime	yes	Production time
`proposer`	address	yes	Proposing validator
`transactionCount`	integer	yes	Number of transactions
`transactions`	array	yes	Hashes, or Transaction objects if requested
`gasUsed`	uint256	yes	Gas consumed
`gasLimit`	uint256	yes	Block gas capacity
`baseFeePerGas`	uint256	yes	Base fee in sparks per gas
`finalized`	boolean	yes	Whether final

13.5 Transaction

Field	Type	Req	Description
`id`	string	yes	Platform ID (`txn_...`)
`hash`	hash	yes	Chain transaction hash
`status`	string	yes	`pending` , `included` , `finalized` , `failed` , `dropped` , `replaced`
`from`	address	yes	Sender
`to`	address	no	Recipient or contract. Absent for deployments.
`value`	uint256	yes	Sparks transferred
`data`	hex	no	Call data, if any

Field	Type	Req	Description
`nonce`	integer	yes	Sender nonce
`gasLimit`	uint256	no	Limit set when signed
`maxFeePerGas`, `maxPriorityFeePerGas`	uint256	no	Fee settings when signed
`gasUsed`	uint256	no	Gas consumed. Present once included.
`effectiveGasPrice`	uint256	no	Price paid per gas unit. Present once included.
`fee`	uint256	no	`gasUsed x effectiveGasPrice`. Present once included.
`blockNumber`, `blockHash`	integer, hash	no	Present once included
`contractAddress`	address	no	For finalized deployments
`revertReason`	string	no	For `failed` transactions, decoded when the ABI is registered
`replacedBy`	string	no	For `replaced`: the ID of the replacement
`events`	array of EventLog	no	Logs emitted, decoded when possible
`metadata`	object	no	Your key-value pairs from submission
`submittedAt`, `includedAt`, `finalizedAt`	datetime	no	Timestamps for each stage reached

13.6 PreparedTransaction

Field	Type	Req	Description
`chainId`	integer	yes	Network the transaction is valid on
`type`	integer	yes	Transaction format (2)

Field	Type	Req	Description
`nonce`	integer	yes	Nonce to sign
`from`	address	yes	Sender
`to`	address or null	yes	Recipient; `null` for deployments
`value`	uint256	yes	Sparks
`data`	hex	yes	Call data (`0x` for transfers)
`gasLimit`	uint256	yes	Gas limit
`maxFeePerGas`	uint256	yes	Fee ceiling
`maxPriorityFeePerGas`	uint256	yes	Tip
`estimatedFee`	uint256	yes	Expected fee
`maxFee`	uint256	yes	`gasLimit x maxFeePerGas`
`unsignedTransaction`	hex	yes	Encoded transaction to sign
`signingHash`	hash	yes	Hash to sign with a KMS or hardware signer
`expiresAt`	datetime	yes	Staleness time for the fee suggestions

13.7 GasEstimate

Field	Type	Req	Description
`gasLimit`	uint256	yes	Estimated gas
`baseFee`	uint256	yes	Current base fee
`suggested`	object	yes	Keys `slow` , `normal` , `fast` , each a **FeeSuggestion**
`blockNumber`	integer	yes	Block estimated against

FeeSuggestion: `maxFeePerGas` (uint256), `maxPriorityFeePerGas` (uint256), `estimatedWaitSeconds` (integer).

13.8 Contract

Field	Type	Req	Description
`address`	address	yes	Contract address
`name`	string	no	Display name
`abiVersion`	integer	yes	Increments each time the ABI is replaced
`deployedAt`	datetime	no	Deployment time, if known
`deploymentTransaction`	string	no	Platform ID of the deploying transaction
`verified`	boolean	yes	Whether verified source matches the bytecode
`indexing`	object	yes	`status` (`catching_up` or `synced`), `lastIndexedBlock` (integer), `startBlock` (integer)
`abi`	array	no	The ABI, when `includeAbi=true`

13.9 ContractCallResult

Field	Type	Req	Description
`function`	string	yes	Function called
`result`	any	yes	Decoded return value: scalar, array or object. Large integers as strings.
`blockNumber`	integer	yes	Block read at

13.10 SimulationResult

Field	Type	Req	Description
`success`	boolean	yes	Whether the call would succeed
`gasUsed`	uint256	yes	Gas the call would use

Field	Type	Req	Description
`result`	any	no	Return value, if successful
`events`	array	no	Decoded events the call would emit
`revertReason`	string	no	Why it would revert, if unsuccessful
`blockNumber`	integer	yes	Block simulated against

13.11 EventLog

Field	Type	Req	Description
`address`	address	yes	Emitting contract
`event`	string	no	Event name (absent if not decodable)
`args`	object	no	Decoded arguments
`topics`	array of hash	no	Raw topics, present when the event is not decoded
`data`	hex	no	Raw data, present when the event is not decoded
`blockNumber`	integer	yes	Block containing the log
`blockHash`	hash	yes	Hash of that block
`transactionHash`	hash	yes	Producing transaction
`logIndex`	integer	yes	Position within the block
`timestamp`	datetime	yes	Block timestamp
`finalized`	boolean	yes	Whether final
`removed`	boolean	no	`true` if retracted (WebSocket `included` subscriptions only)

13.12 Token

Field	Type	Req	Description
`address`	address	yes	Token contract
`type`	string	yes	`cft20` or `cft721`
`name`, `symbol`	string	yes	Display values
`decimals`	integer	CFT-20	0 to 18
`totalSupply`	uint256	yes	Current supply
`maxSupply`	uint256	no	Hard cap, if any
`owner`	address	no	Administrating account
`features`	object	no	`mintable`, `burnable`, `pausable`, `allowList` (booleans)
`paused`	boolean	no	Whether transfers are frozen
`baseUri`	string	CFT-721	Metadata URI prefix
`blockNumber`	integer	yes	Block read at

13.13 NftItem

Field	Type	Req	Description
`collection`	address	yes	Collection contract
`tokenId`	string	yes	Token ID
`tokenUri`	string	yes	On-chain metadata location
`name`, `description`, `image`	string	no	From metadata
`attributes`	array	no	Objects with `trait_type` and `value`
`metadataStatus`	string	yes	`ok`, `pending`, `unreachable` or `invalid`

13.14 Webhook

Field	Type	Req	Description
`id`	string	yes	`wh_...`
`url`	string	yes	Delivery URL
`description`	string	no	Free text
`events`	array of string	yes	Subscribed event types
`filters`	object	no	`addresses`, `contracts`, `tokens`, `eventNames`, `finality`
`headers`	object	no	Custom headers (values are masked in responses)
`enabled`	boolean	yes	Delivery on or off
`secret`	string	creation and rotation only	Signing secret, shown once
`health`	object	no	`status`, `lastSuccessAt`, `lastFailureAt`, `consecutiveFailures`
`disabledReason`	string	no	Why the platform disabled it
`createdAt`, `updatedAt`	datetime	yes	Timestamps

13.15 WebhookEvent

Field	Type	Req	Description
`id`	string	yes	`evt_...`; unique per event, constant across retries and redeliveries
`type`	string	yes	Event type (section 10.9)
`apiVersion`	string	yes	Payload version

Field	Type	Req	Description
`createdAt`	datetime	yes	Creation time
`network`	string	yes	`mainnet`, `ember` or `devnet`
`data`	object	yes	Type-specific payload

13.16 WebhookDelivery

Field	Type	Req	Description
`eventId`	string	yes	The event
`eventType`	string	yes	Its type
`status`	string	yes	`succeeded`, `failed`, `pending`
`attempts`	array	yes	Objects: `attempt`, `sentAt`, `responseStatus`, `durationMs`, `error`
`nextAttemptAt`	datetime or null	yes	Next retry time

13.17 Pagination

Field	Type	Req	Description
`data`	array	yes	Items on this page
`nextCursor`	string or null	yes	Cursor for the next page; `null` when finished

13.18 Usage

Field	Type	Req	Description
`from`, `to`	datetime	yes	Period
`granularity`	string	yes	`minute`, `hour` or `day`

Field	Type	Req	Description
`plan`	string	yes	`free`, `growth`, `enterprise`
`limits`	object	yes	`requestsPerSecond`, `requestsPerDay`, `webSocketSubscriptions`
`totals`	object	yes	`requests`, `errors`, `rateLimited`, `errorRate`
`series`	array	yes	Buckets: `start`, `requests`, `errors`, `rateLimited`, `p95LatencyMs`, optional `group`
`quota`	object	yes	`dailyUsed`, `dailyRemaining`, `resetsAt`

13.19 Error

Field	Type	Req	Description
`error.code`	string	yes	Stable machine-readable code (chapter 14)
`error.message`	string	yes	Human-readable text
`error.details`	object	no	Structured data; keys depend on `code`
`error.requestId`	string	yes	The request's ID
`error.docUrl`	string	no	Link to the catalog entry

Common `details` shapes

Code	`details`
`validation_failed`	`{ "fields": [ { "field": "to", "issue": "..." } ] }`
`insufficient_funds`	`{ "required": "uint256", "available": "uint256" }`
`execution_reverted`	`{ "revertReason": "string" }`
`already_known`	`{ "transaction": { Transaction } }`
`rate_limited`	`{ "retryAfter": 2 }`

Code	`details`
`invalid_address`	`{ "addresses": ["0x..."] }` (for batch requests)

---

14. Error Catalog

This chapter lists every error `code` the REST API can return. Write client code that branches on `code`, never on `message`. For the error object's structure see sections 3.2 and 13.19.

14.1 Errors by HTTP Status

Status	Name	Meaning	Retry?
`400`	Bad Request	The request is malformed or a value has the wrong format	No. Correct the request.
`401`	Unauthorized	Credentials are missing, invalid or expired	No. Fix the credentials (or refresh the token, then retry once).
`403`	Forbidden	The credential is valid but not allowed to do this	No. Change scopes, permission or IP rules.
`404`	Not Found	The resource does not exist (or belongs to another project)	Only for something created within the last few seconds. See below.
`409`	Conflict	The request conflicts with current state	Depends on the `code`.
`422`	Unprocessable	The request is well-formed but cannot be carried out, often because the call would revert on-chain	No. Correct the input or the state.
`429`	Too Many Requests	Rate limit exceeded	Yes, after `Retry-After`.

Status	Name	Meaning	Retry?
`500`	Internal Server Error	Unexpected platform error	Yes, with backoff. Report persistent cases with the `requestId`.
`502`	Bad Gateway	An upstream node returned an invalid response	Yes, with backoff.
`503`	Service Unavailable	Temporary overload or maintenance	Yes, after `Retry-After` if present.
`504`	Gateway Timeout	An upstream node did not respond in time	Yes, with backoff. For `POST /transactions`, **reuse the same `Idempotency-Key`**.

The `404` and eventual consistency

After you create a resource, a read routed to a different API server can briefly return `404`. For an ID or hash you received moments ago, retry up to 5 times with 500 ms spacing before treating the `404` as real.

Retrying safely

Situation	Safe to retry?
`GET` requests	Always
`POST /transactions` with an `Idempotency-Key`	Always
`POST /tokens`, `POST /contracts` with an `Idempotency-Key`	Always
`POST` requests **without** an idempotency key	Only if you can verify nothing was created
`PATCH` and `DELETE`	Yes (repeating a `DELETE` returns `404`, which means the goal is met)

14.2 Error Codes

Authentication and authorization

Code	HTTP	Cause	Remedy
`missing_credentials`	401	No `Authorization` header, or it is not `Bearer <token>`	Send `Authorization: Bearer <key>`
`invalid_api_key`	401	Wrong, revoked or deleted key; or a key from another project	Re-copy the key; check the prefix matches the intended network; create a new key
`token_expired`	401	OAuth access token older than one hour	Request a new token (section 2.4)
`insufficient_scope`	403	The credential lacks the scope the endpoint needs. `details.required` names it.	Create a key with the right permission, or request the scope
`ip_not_allowed`	403	Request came from an address outside the key's allow-list	Add the address in the Console, or call from an allowed host

Request format and validation

Code	HTTP	Cause	Remedy
`validation_failed`	400	One or more fields invalid. `details.fields` lists them.	Fix each listed field
`invalid_address`	400	Not 42 hex characters, or the checksum is wrong	Validate with the SDK (`isAddress`) before sending
`invalid_amount`	400	Not an integer string, negative, or beyond 78 digits	Send integer sparks (or token units) as strings

Code	HTTP	Cause	Remedy
`invalid_block`	400	`block` is not `latest`, `finalized`, `pending` or a number	Correct the selector
`invalid_block_id`	400	`blockId` path value is not a number, hash, `latest` or `finalized`	Correct the identifier
`invalid_block_range`	400	`fromBlock` is after `toBlock`	Swap or correct the range
`block_range_too_large`	400	More than 100,000 blocks requested	Split into smaller ranges
`invalid_cursor`	400	Cursor expired, malformed, or used with different filters	Restart pagination from the first page
`invalid_status`	400	Unknown `status` filter	Use a value from section 15.1
`invalid_event_type`	400	Unknown webhook event type	Use a type from section 10.9
`invalid_time_range`	400	`from` after `to`, or range too long for the granularity	Shorten the range or coarsen the granularity
`too_many_addresses`	400	More than 50 addresses in a batch	Split into batches of 50
`invalid_signed_transaction`	400	The signed payload cannot be decoded or the signature is invalid	Re-sign the exact bytes returned by `prepare`
`invalid_bytecode`	400	Bytecode is empty or not valid hex	Provide compiled creation bytecode
`invalid_abi`	400	ABI is not a valid JSON array of ABI items	Use the compiler's ABI output unchanged

Code	HTTP	Cause	Remedy
`missing_idempotency_key`	400	An endpoint requiring `Idempotency-Key` was called without it	Add the header

Not found

Code	HTTP	Cause	Remedy
`transaction_not_found`	404	Unknown platform ID or hash	Check the ID; retry briefly if just created
`block_not_found`	404	Block number is above the latest block, or the hash is unknown	Check the network and number
`contract_not_found`	404	No code exists at the address on this network	Confirm network and address
`contract_not_registered`	404	The contract has no ABI registered in your project	Call `POST /contracts/{address}/register`
`token_not_found`	404	No contract at the address	Confirm network and address
`webhook_not_found`	404	Unknown webhook ID, or it belongs to another project	List webhooks and use a valid ID
`delivery_not_found`	404	Unknown event, or older than the 30-day history	Use a recent event ID

Conflicts

Code	HTTP	Cause	Remedy
`already_known`	409	The identical signed transaction is already in the mempool. `details.transaction` holds it.	Track the existing transaction; nothing more to do
`idempotency_conflict`	409	The `Idempotency-Key` was used earlier with a different request body	Use a new key for a new operation; replay the **same** body for a retry
`idempotency_in_progress`	409	The first request with this key is still being processed	Wait a second and retry with the same key
`transaction_not_pending`	409	Speed-up or cancel attempted on a transaction that is no longer pending	Fetch the transaction and act on its current status
`rotation_in_progress`	409	A previous secret rotation's 24-hour overlap has not ended	Wait until `previousSecretExpiresAt`
`webhook_disabled`	409	The webhook is disabled	Re-enable it with `PATCH` before redelivering

Transaction and chain state (422)

Code	Cause	Remedy
`wrong_network`	The signed transaction's chain ID differs from your key's network	Sign with the chain ID from `prepare` (7700, 7701 or 7702)
`nonce_too_low`	The nonce was already used	Fetch the nonce with `block=pending`
`nonce_gap`	A lower nonce is missing, so this transaction cannot be included yet	Send the missing nonce, or cancel and renumber
`insufficient_funds`	Balance below `value + gasLimit x maxFeePerGas`. `details` has `required` and `available`.	Fund the account or lower the fee ceiling

Code	Cause	Remedy
`fee_too_low`	`maxFeePerGas` is below the current base fee	Re-run `prepare` or `gas/estimate`
`fee_bump_too_low`	A replacement's fee is less than 10 percent above the original	Raise both fee values by at least 10 percent
`gas_limit_too_low`	The limit cannot cover even the intrinsic gas	Use the estimated limit with a buffer
`execution_reverted`	The call would revert. `details.revertReason` explains.	Fix the call or the contract state; simulate first
`state_unavailable`	Historical state is not available for the requested block on your plan	Use a more recent block or an archive-enabled plan

Contracts and tokens (422)

Code	Cause	Remedy
`bytecode_too_large`	Bytecode exceeds 256 KB	Reduce contract size
`constructor_args_mismatch`	Arguments do not match the constructor in the ABI	Provide the right count and types
`unknown_function`	No such function in the registered ABI	Check the name; use the full signature for overloads
`function_not_readonly`	`call` was used for a function that changes state	Send a transaction instead
`invalid_arguments`	Argument types do not match the function. `details.expected` lists types.	Correct the arguments
`unknown_event`	Event name is not in the ABI	Check the name
`unknown_argument`	Filter on an argument that is not indexed	Filter only on indexed arguments
`not_a_token`	The contract does not implement CFT-20 or CFT-721	Use the contract endpoints instead

Code	Cause	Remedy
`invalid_decimals`	`decimals` is outside 0 to 18	Choose a valid value
`initial_supply_exceeds_cap`	`initialSupply` is greater than `maxSupply`	Lower the supply or raise the cap
`feature_unsupported`	A feature flag is not available for this token type	Remove the flag

Accounts and webhooks (422)

Code	Cause	Remedy
`metadata_too_large`	More than 16 keys, or a key or value is too long	Shrink the metadata
`account_limit_reached`	Registered-address limit of your plan reached	Remove unused accounts or upgrade
`webhook_unreachable`	The verification ping did not return `2xx` within 10 seconds	Make the endpoint public, HTTPS and fast
`insecure_url`	Not HTTPS (live projects), or the host resolves to a private network address	Use a public HTTPS URL
`webhook_limit_reached`	Active-webhook limit of your plan reached	Delete or disable unused webhooks

Rate limiting

Code	HTTP	Cause	Remedy
`rate_limited`	429	Per-second or per-day limit exceeded	Wait `Retry-After`; back off with jitter
`redelivery_rate_limited`	429	More than 60 webhook redeliveries per minute	Slow down

Server errors

Code	HTTP	Cause	Remedy
`internal_error`	500	Unexpected platform error	Retry with backoff; contact support with the `requestId` if it persists
`bad_gateway`	502	Upstream node error	Retry with backoff
`service_unavailable`	503	Overload or maintenance	Retry after `Retry-After`
`gateway_timeout`	504	Upstream node timeout	Retry; reuse idempotency keys for `POST`

---

15. Reference Tables

15.1 Enumerations

Transaction status

Value	Terminal?	Description
`pending`	No	In the mempool
`included`	No	In a block that is not yet final
`finalized`	Yes	In a final block and succeeded
`failed`	Yes	Executed and reverted
`dropped`	Yes	Left the mempool without inclusion
`replaced`	Yes	Superseded by another transaction with the same nonce

Networks

network	environment	Chain ID	Token
`mainnet`	`live`	7700	FORGE
`ember`	`test`	7701	tFORGE
`devnet`	`test`	7702	dFORGE

Fee speeds

Value	Typical wait	Use for
`slow`	About 12 seconds	Batch and non-urgent work
`normal`	About 4 seconds	Default
`fast`	About 2 seconds	User is waiting; replacements

Account type

`eoa`, `contract`

Token type

`cft20` (fungible), `cft721` (non-fungible)

Metadata status

`ok`, `pending`, `unreachable`, `invalid`

Webhook event types

`webhook.ping`, `transaction.pending`, `transaction.included`, `transaction.finalized`, `transaction.failed`, `transaction.dropped`, `account.received`, `token.transfer`, `contract.event`, `contract.deployed`

WebSocket subscription types

`block.new`, `transaction.status`, `account.activity`, `contract.event`, `token.transfer`

Finality levels

`included`, `finalized`

Block selectors

`latest`, `finalized`, `pending` (nonce queries), or a block number

Webhook health status

`healthy`, `degraded`, `disabled`

Plans

`free`, `growth`, `enterprise`

> **Note:** New values may be added to any enumeration within a version. Treat unknown values as non-fatal (section 3.7).

15.2 Limits

Limit	Value
Request body size	1 MB
Signed transaction size	128 KB
Contract bytecode size	256 KB
Page size (`limit`)	1 to 100; default 25
Addresses per batch balance request	50
Block range per event query	100,000 blocks
Historical state without an archive plan	Last 128 blocks
Cursor validity	24 hours
Idempotency key length	8 to 64 characters
Idempotency key retention	24 hours
Transaction metadata	8 string key-value pairs

Limit	Value
Account metadata	16 keys; key up to 40 characters; value up to 256
Account label	64 characters
Token name / symbol	64 / 11 characters
Webhook URL length	2,048 characters
Webhooks per project	20 (Free plan: 3)
Custom headers per webhook	5
Webhook response timeout	10 seconds
Webhook delivery attempts	7
Webhook delivery history	30 days
Webhook timestamp tolerance	5 minutes
WebSocket message size	64 KB
WebSocket idle timeout	60 seconds
WebSocket replay window	10,000 blocks
Access token lifetime	3,600 seconds
Mempool expiry for a transaction	30 minutes
Rate limit (Free / Growth)	10 / 50 requests per second
Daily quota (Free / Growth)	100,000 / 5,000,000 requests

15.3 Network Parameters and Chain IDs

Parameter	Forge Mainnet	Ember Testnet	Local Devnet
Chain ID	7700	7701	7702
Native token	FORGE	tFORGE	dFORGE
Block time	About 2 s	About 2 s	Configurable (default instant)
Time to finality	About 4 s	About 4 s	Instant

Parameter	Forge Mainnet	Ember Testnet	Local Devnet
JSON-RPC	`https://rpc.chainforge.example`	`https://rpc.ember.chainforge.example`	`http://127.0.0.1:8545`
WebSocket	`wss://ws.chainforge.example/v1`	`wss://ws.ember.chainforge.example/v1`	`ws://127.0.0.1:8545`
Explorer	`https://explorer.chainforge.example`	`https://explorer.ember.chainforge.example`	Built-in
Faucet	None	`https://faucet.ember.chainforge.example`	Pre-funded accounts

Protocol constant	Value
Smallest unit	1 spark = 10^-18 FORGE
Minimum gas for a transfer	21,000
Block gas limit	30,000,000
Transaction type	2 (fee-market transactions)
Address length	20 bytes (42 characters with `0x`)
Hash length	32 bytes (66 characters with `0x`)
Zero address	`0x0000000000000000000000000000000000000000`

---

16. Changelog and Deprecations

16.1 API Changelog

Changes are grouped as **Added**, **Changed**, **Deprecated**, **Removed**, **Fixed** and **Security**. Breaking changes are marked **BREAKING** and occur only in major versions.

> **Note:** The entries below are fictional and demonstrate the format. They are consistent with the Developer Guide's release notes (Appendix C).

v1.4.0 (October 2026)

Added

- `POST /transactions/{id}/speed-up` and `POST /transactions/{id}/cancel` return prepared replacement transactions.

- `GET /blocks/finalized` shortcut.
- `fromBlock` and `finality` options on `GET /contracts/{address}/events`.

Changed

- `POST /webhooks/{id}/rotate-secret` now keeps the previous secret valid for 24 hours and signs with both.
- Revert reasons for custom Solidity errors are decoded when the contract is registered.

Fixed

- `GET /accounts/{address}/transactions` no longer omits `replaced` transactions when `status` is not specified.

v1.3.0 (September 2026)

Added

- `GET /accounts/{address}/nfts` with collection filters and `metadataStatus`.
- WebSocket `finality` option and cursor-based `resumeFrom`.
- `GET /usage` with `groupBy`.

Changed

- Free-plan rate limit raised from 5 to 10 requests per second.

v1.2.0 (July 2026)

Added

- Webhook delivery log (`GET /webhooks/{id}/deliveries`) and manual redelivery.
- Cursor pagination on all list endpoints.
- OAuth 2.0 client-credentials authentication and fine-grained scopes.

Deprecated

- `GET /accounts/{address}/history` (use `GET /accounts/{address}/transactions`). See 16.2.

v1.1.0 (May 2026)

Added

- `POST /tokens` and token read endpoints.

- `Idempotency-Key` header on all state-changing endpoints.

v1.0.0 (March 2026)

- General availability of the REST API for Forge Mainnet and Ember.

16.2 Deprecated Endpoints and Sunset Dates

Endpoint	Deprecated in	Sunset (removal) date	Replacement	Migration notes
`GET /accounts/{address}/history`	v1.2.0 (July 2026)	31 March 2027	`GET /accounts/{address}/transactions`	The new endpoint uses `cursor` instead of `page`, and returns the **Transaction** object. Replace `page` and `pageSize` with `cursor` and `limit`.

How you will know. Deprecated endpoints return `Deprecation: true` and a `Sunset` header (section 3.7). The Console shows a warning for any key that called a deprecated endpoint in the past 30 days, and administrators receive an email 90, 30 and 7 days before removal.

Version policy reminder. A previous major version is supported for at least 12 months after its successor is generally available.

Appendices

Appendix A: Code Samples: Your First Request

These samples call two endpoints: `GET /status` (to verify connectivity and the network) and `GET /accounts/{address}/balance`. Set `CHAINFORGE_API_KEY` in your environment first.

curl

```
export CHAINFORGE_API_KEY="cfk_test_YOUR_KEY"
ADDRESS="0x71C7656EC7ab88b098defB751B7401B5f6d8976F"

curl -s https://api.chainforge.example/v1/status \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"

curl -s "https://api.chainforge.example/v1/accounts/$ADDRESS/balance?
block=finalized" \
  -H "Authorization: Bearer $CHAINFORGE_API_KEY"
```

JavaScript (Node.js 18+, no SDK)

```
const BASE = "https://api.chainforge.example/v1";
const headers = { Authorization: `Bearer ${process.env.CHAINFORGE_API_KEY}`
};

async function get(path) {
  const res = await fetch(`${BASE}${path}`, { headers });
  const body = await res.json();
  if (!res.ok) {
    const { code, message, requestId } = body.error;
    throw new Error(`${res.status} ${code}: ${message} (${requestId})`);
  }
  return body;
}

const status = await get("/status");
if (status.chainId !== 7701) throw new Error(`Unexpected chain
${status.chainId}`);

const bal = await
get("/accounts/0x71C7656EC7ab88b098defB751B7401B5f6d8976F/balance?
block=finalized");
console.log(`${bal.balanceFormatted} ${bal.symbol} at block
${bal.blockNumber}`);
```

JavaScript (SDK)

```
import { ChainForge } from "@chainforge/sdk";

const client = new ChainForge({ apiKey: process.env.CHAINFORGE_API_KEY });
const status = await client.status.get();
const bal = await
client.accounts.getBalance("0x71C7656EC7ab88b098defB751B7401B5f6d8976F", {
block: "finalized" });
console.log(status.network, bal.balanceFormatted, bal.symbol);
```

Python (requests, no SDK)

```
import os, requests

BASE = "https://api.chainforge.example/v1"
HEADERS = {"Authorization": f"Bearer {os.environ['CHAINFORGE_API_KEY']}"}

def get(path, **params):
    r = requests.get(BASE + path, headers=HEADERS, params=params, timeout=15)
    body = r.json()
    if not r.ok:
        e = body["error"]
        raise RuntimeError(f"{r.status_code} {e['code']}: {e['message']}
({e['requestId']})")
    return body

status = get("/status")
assert status["chainId"] == 7701, f"Unexpected chain {status['chainId']}"

bal = get("/accounts/0x71C7656EC7ab88b098defB751B7401B5f6d8976F/balance",
block="finalized")
print(bal["balanceFormatted"], bal["symbol"], "at block", bal["blockNumber"])
```

Python (SDK)

```
import os
from chainforge import ChainForge

client = ChainForge(api_key=os.environ["CHAINFORGE_API_KEY"])
status = client.status.get()
bal =
client.accounts.get_balance("0x71C7656EC7ab88b098defB751B7401B5f6d8976F",
block="finalized")
print(status.network, bal.balance_formatted, bal.symbol)
```

What a successful run prints

```
94.999937 tFORGE at block 4820141
```

Handling errors in all languages

1. Check the HTTP status.
2. On failure, read `error.code`, `error.message` and `error.requestId`.
3. Decide to retry (`429`, `5xx`) or fix the request (`4xx`) using the table in section 14.1.
4. Log the `requestId`.

Appendix B: Postman Collection and OpenAPI Download

The API is described by an **OpenAPI 3.0** specification, which is the source of truth for this guide.

Resource	Location
OpenAPI 3.0 specification (YAML)	`https://docs.chainforge.example/openapi/v1/chainforge.yaml`
OpenAPI 3.0 specification (JSON)	`https://docs.chainforge.example/openapi/v1/chainforge.json`
Postman collection (v2.1)	`https://docs.chainforge.example/postman/chainforge-v1.postman_collection.json`

Resource	Location
Postman environments	`ember.postman_environment.json`, `mainnet.postman_environment.json`

> **Note:** These locations are fictional placeholders.

Using the OpenAPI specification

Goal	How
Explore interactively	Open the file in Swagger Editor or Swagger UI
Browse as reference docs	Render with Redoc or Redocly
Mock the API locally	`npx @stoplight/prism-cli mock chainforge.yaml`
Generate a client library	Use OpenAPI Generator for your language
Lint it	`npx @redocly/cli lint chainforge.yaml`
Detect changes between versions	`oasdiff breaking old.yaml new.yaml`

Using the Postman collection

1. In Postman, choose **Import** and select the collection file.
2. Import the `ember` environment file.
3. Open the environment and set `apiKey` to your `cfk_test_` key. The collection sends it as the bearer token.
4. Run **Status → Get status** to confirm that the key and network are correct.
5. Work through the folders in order: Accounts, Transactions, Contracts, Tokens, Webhooks.

The collection includes pre-request scripts that generate a fresh `Idempotency-Key` for every state-changing request and tests that check the response against the documented schema.

How an endpoint looks in the specification

An excerpt for `GET /accounts/{address}/balance`:

```
paths:
  /accounts/{address}/balance:
    get:
      tags: [Accounts]
      operationId: getAccountBalance
      summary: Return an account's native balance and nonce
      security:
        - bearerAuth: []
        - oauth2: [accounts:read]
      parameters:
        - name: address
          in: path
          required: true
          description: Account address.
          schema:
            type: string
            pattern: '^0x[0-9a-fA-F]{40}$'
        - name: block
          in: query
          description: Block selector. One of latest, finalized, or a block
number.
          schema:
            type: string
            default: latest
      responses:
        '200':
          description: The account balance.
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Balance'
        '400':
          $ref: '#/components/responses/BadRequest'
        '401':
          $ref: '#/components/responses/Unauthorized'
        '429':
          $ref: '#/components/responses/TooManyRequests'
```

Appendix C: Glossary

The full glossary is Appendix A of the ChainForge Developer Guide. These are the terms most often needed while using this reference.

Term	Definition
ABI	JSON description of a contract's functions, events and errors
Address	42-character identifier of an account, beginning `0x`
Base fee	Protocol-set minimum price per unit of gas; burned
Block selector	The `block` parameter choosing which chain state to read (`latest`, `finalized`, number)
Chain ID	Number identifying a network; prevents cross-network replay (7700, 7701, 7702)
Cursor	Opaque token for fetching the next page of a list
EOA	Externally owned account, controlled by a private key
Event log	A record emitted by a contract during a transaction
Finality	The point at which a block can no longer be reversed (about 4 seconds on ChainForge)
Gas / gas limit	Unit of computational work / maximum a transaction may consume
Idempotency key	Unique value making repeated requests produce one effect
Mempool	Waiting area for valid transactions not yet in a block
Nonce	Per-account transaction counter for ordering and replay protection
Prepared transaction	An unsigned transaction returned by the API for you to sign

Term	Definition
Revert	Undoing of all changes made by a failed contract call
Scope	A permission attached to a credential (for example, `transactions:write`)
Spark	Smallest unit of FORGE; 1 FORGE = 10^18 sparks
Webhook	An HTTPS request sent to your server when an event occurs

---

End of the ChainForge API Reference Guide.